What Commit-Time Isolation Leaves Behind: Mechanized Anomaly Predicates, Verified Disciplines, and the Residue in Multi-Agent LLM Systems

Sajjad Khan

Abstract—An agent's step in a multi-agent LLM system is a transaction whose compute phase is an inference call of unbounded latency, during which the state it read is neither locked nor re-validated. We give the regime a mechanized consistency theory and test what it buys. Four classical conditions become history predicates—stale generation, phantom tool, causal cascade, tool-effect reordering—with TLC witnesses; TLAPS proves that stale-generation freedom implies commit-order read consistency under three hypotheses, which gates check on the committed records; bounded model checking realizes all sixteen points of their exclusion lattice; and Verus verifies one chain of disciplines whose verified files are byte-identical to the runtime that was measured. The empirical result is what commit-time isolation cannot reach. On a released LangGraph runtime with live models, its store instrumented only to stamp a clock, read-set validation removes every commit grounded in a superseded store read, yet the booking contradicts the final trip date in 1–65% of sessions per family and arm, because the revision lands after the commit—a harm that occurs in correct sequential executions. We define it as a predicate on effects and their recovered bases, show that under an open-world store it can only be owed, and test a basis ledger that registers the obligation with the superseding write against a simulated world with refusals and a downstream consumer: the ledger's report and the world agree in $300/300$ sessions, a compensator that misreported was caught only by the world check, and a consumer still acts on the stale booking until the compensation's model call completes. A released tool executor commits an order-dependent pipeline out of order on 22 of 30 turns of one backbone while its transcript shows it in order. Per sound committed operation, snapshot isolation costs $+0.4$–11.1% in tokens and pessimistic locking up to $+97\%$; per session, a model told the truth about a refusal makes 2–4$\times$ the calls on the contended workloads when the refusal names no agent and 1.1–2.3$\times$ when it does; a census of 98 public LangGraph repositories finds the precondition's statically visible shape, a co-scheduled reader and writer of one channel, confirmed nowhere. From these we derive four concrete requirements for agent frameworks. Every number carries its receipt and its assurance level.

Index Terms—Multi-agent systems, large language models, memory consistency, isolation levels, formal verification, TLA+, Verus.

1 Introduction

A multi-agent LLM system shares state: one agent writes a plan that another reads, a tool registry is consulted at planning time and called at execution time, a supervisor retracts a decision a worker has already acted on. Each agent's step has the shape of a transaction—read, compute, commit—with one difference from every setting in which transactions have been studied: the compute phase is an inference call that takes seconds to minutes, during which the read set is neither locked nor cheaply re-validated. Deployed frameworks place this phase inside a pinned snapshot (LangGraph's superstep), a re-rendered context (Letta) or a message queue (AutoGen), and none states what guarantee the shared state carries across it.

1.1 A failure, constructed and then observed

Two agents share a trip record. A flight agent reads "trip date = 14 June," drafts a reservation over a model call, and books it. During that call a second agent, acting for the user, moves the trip to 21 June. The flight agent commits a 14 June booking beside a 21 June trip date. No component failed; the history is even serializable, with the flight agent first and the revision after. What it is not is serializable *in the order in which its operations committed*: the revision committed first, and a flight agent executing after it would have read 21 June (Figure 1).

We ran this scenario on a released LangGraph runtime under live models, its store instrumented only to stamp a logical clock (Section 6.1). With the second agent started inside the first agent's generation window, the framework committed the stale booking in every session in which the revision landed inside the window—60% and 91% of sessions on two API models—and read-set validation at commit removed every such firing. The same data record a fact the predicate does not see: when the revision lands *after* the commit, the booking contradicts the final trip date just the same, and no isolation discipline can prevent it. We report both numbers throughout, because their distance is the distance between a consistency predicate and the harm it is meant to track; the rates are properties of the race we opened (150 ms offset; swept in Figure 2) and carry no prevalence reading.

What this paper is not claiming. A census of 98 public LangGraph repositories (Section 6.7) finds the statically visible form of A_1's precondition—a reader and a writer of one last-value channel scheduled together inside a graph—confirmed in none of 407 graph sites, eight unresolved; the cross-thread store the failure above shares is not visible to a static front end and is not counted. We take the census at face value: public LangGraph code is mostly sequential, and four of five frameworks we probed close the window by construction. The regime studied is the one in which contention

S. Khan is an independent researcher (e-mail: sajjadanwar200@gmail.com).
Artifact: https://github.com/sajjadanwar0/mac-consistency-mono, tag `v4.1-tse`*, cut by a gate that refuses while any cell of this text is marked pending: TLA+ models, Verus proofs, the runtime, every harness and the committed traces behind every table;* `reproduce.sh` *re-derives each figure, and every stated number is in* `CLAIMS.json` *or regenerated into* `claims_results.tex` *from a committed receipt. A supplement (§§A–W) holds raw tables and proofs omitted for length.*

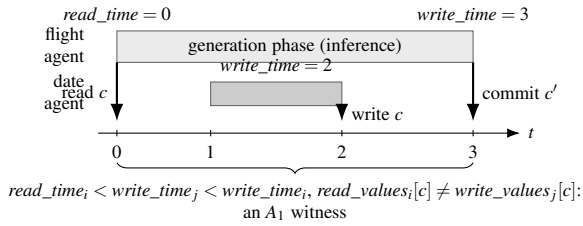


Fig. 1. The read–generate–commit window. Logical time is the length of the operation log. Operation i (flight agent) reads c at $read_time_i = 0$; operation j (date agent) commits a different value to c at $write_time_j = 2$, inside i's window; i commits at $write_time_i = 3$ grounded in the superseded read (L_1 re-validates the read set at $write_time_i$ and discards the generation).

is designed in (parallel fan-out over a shared store, several agents for one user, a registry mutated in flight); the contribution is not that the anomalies are common but mechanized definitions, disciplines priced per unit of sound work, and an account of what each leaves behind.

1.2 Why existing theory does not operationalize these failures

Hardware models [5], [6] presuppose bounded latency and static access sets; database isolation theory [7]–[9] supplies the anomaly-by-anomaly method we follow but holds the read set under lock or snapshot for the transaction's lifetime; replicated-storage consistency [10] treats operations as discrete events with summarizable signatures. The agent regime has unbounded generation latency, a mutable tool registry and irreversible effects, so the cascading-abort machinery of database recovery, which assumes effects can be undone, is exactly what fails.

1.3 Contributions

We claim no new concurrency theory; each anomaly below is a classical one, and we say which. The contribution is the operational port of four of them to agent runtimes, its mechanization at stated assurance levels, and an evaluation that measures the gap between each predicate and the harm it tracks, including the part of the harm that is not a concurrency anomaly.

(C1) A catalog with classical names, and what it leaves out (Section 3). Four history predicates in TLA$^+$—stale generation (A_1), phantom tool (A_2), causal cascade (A_3), tool-effect reordering (A_6)—each with a TLC witness and its classical counterpart: a failed backward validation [4], a dangling capability reference, recoverability with externalization as the commit point [16], and order preservation of external effects. Theorem 1 (TLAPS, 60 obligations) places A_1-freedom among the classical classes exactly, a conditional guarantee whose hypotheses (per-agent non-overlap, read-before-commit, faithful records) are stated, checked on every committed record, and violated by one deployed construct we name. Two predicates stand outside the catalog, each for a stated reason: the *residue predicate R*—an irreversible effect is out while ordinary later writes have revised the state it was derived from; not an anomaly (it fires in correct sequential executions) but the harm no commit-time discipline reaches, decided on the records by argument provenance; on the live study A_1 and R partition the outcome column (Section 3.6)—and the *duplication predicate D*—one node's irreversible effect out twice, the shape a deployed resumption path produces, with its witness and the guard that excludes it (Section 3.7).

(C2) A lattice, realized at stated bounds (Section 4). The 2^4 exclusion lattice is trivial; what is not is that every point is inhabited by a checked runtime configuration that excludes exactly its anomalies (64 TLC runs, no mismatch, at two agents, two cells, two operations and, for the safety half, two values; the NoA1 half rechecked at three operations), that one chain $L_0 \subsetneq \cdots \subsetneq L_4$ has a verified realizing runtime at every level, and that the basis ledger composes with every point without disturbing its verdict (160 further runs).

(C3) Verified disciplines, verified detectors, and exactly which bytes were measured (Section 5). Under two enumerated axioms and three enumerated stubs, with no `assume` or `admit`: detectors sound and complete against their specifications, two concurrent L_1 stores proved window-free under a verified reader–writer lock, and L_2, L_3 and L_4 runtimes whose verified files are, byte for byte, the files the runtime compiles (a gate checks the identity) and the bytes measured. The A_1 verdict of every table is the verified detector executing, behind one interning wrapper in the trust base, gated against a hand-written port on every cell; the gate's last pass rescored 144 cells (14,400 traces) and exited non-zero on 18 per-cell refusal sidecars it read as op-records, so its clean receipt is **[pending: runner step 14 rerun]**. Table 1 labels every claim with its assurance level (checked model, verified decision procedure, verified bytes, unverified port, open) and names each correspondence between TLA$^+$ definition, Verus restatement, executed code and scorer, with its hypothesis and gate; of 371 obligations, 236 bear a deployed or measured artifact (supplement § S).

(C4) An evaluation scored on outcomes, not flags, with its own confounds measured (Section 6). For each discipline we report the anomaly it excludes, the harm that remains, and the price: on the live LangGraph study the L_1 residue is R in 1–65% of sessions per family and arm (pooled over three interleaved runs; 0–73 of 100 in single runs), in two implementations of L_1 between which the replications detect no difference; against a simulated world the ledger's report and the world agree in $300/300$ sessions, where a first compensator had misreported 1–10% (Section 6.2); output commit removes exposure exactly when the review precedes the release; per sound committed operation, snapshot isolation costs $+0.4$–11.1% and pessimistic locking up to $+97\%$ in tokens, and, per session, 2–4$\times$ the model calls under a refusal that names no agent and 1.1–2.3$\times$ under one that does (Section 6.6). Where the design limits what a difference can mean we quantify the confound on the receipts.

(C5) Where the preconditions occur (Section 6.7). A census of 98 public LangGraph repositories (407 graph sites) finds no confirmed co-scheduled reader and writer of a last-value channel (eight sites unresolved; the static front end cannot resolve the routing of sites in about half the repositories)—a negative result for A_1's structural precondition, which we report as one—while a released tool executor dispatches a model-emitted, order-dependent pipeline concurrently on $30/30$ turns of one backbone and never on another, which is the case for A_6. **(C6)** Four requirements for agent frameworks follow, each priced (Section 9).

What this paper does not establish. No production prevalence for any anomaly: every occurrence figure is a race our harness opened or a rate on our own workloads, and the census measures how public code is written, not how often it runs. The instrumentation and the Python harnesses are not verified (Section 5.7); the refinement of

the deployed stores to their abstract machines is open; a verified arm's zero is a theorem executing, not evidence.

2 RUNTIME MODEL

A system consists of a finite set of agents A, a shared memory M mapping cells to values with a distinguished sentinel NULL, and a shared registry R of available tools. An agent executes a sequence of *operations*, each in three phases: a *read* phase that captures the values of a chosen subset of cells (the operation's *read_set*) together with the registry contents (*read_registry*), stamping the read with the current log length as *read_time*; a *generation* phase, dominated by inference latency and not treated as instantaneous; and a *write* phase that atomically appends a record to the log, installing the planned writes (*write_set*, *write_values*), the selected tool (*planned_tool*) and the registry at commit (*write_registry*), stamped *write_time*. The log length is a logical clock [13]; a history h is the log, $h \in \mathrm{Seq}(\mathit{OpRecord})$. The specification is `Memory.tla`.

Store axiom. A read returns the latest committed value at or before its read time (stamps are log lengths, so no write shares a stamp with a read and "after" is the strict inequality of Definition 1). TLC confirms this and the two hypotheses below as invariants of `Memory.tla` over all 2,961,434 reachable states at the checked bounds, and gates check all three on the committed records (Section 3.1).

Per-agent non-overlap; read before commit. Two operations of one agent do not overlap in $[\mathit{read_time}, \mathit{write_time}]$; every operation reads strictly before it commits. Both are invariants of the model, and both are load-bearing for Theorem 1 and for the detector transcription of Section 5.1. A framework that runs one node as several parallel instances under one name (LangGraph's `Send` fan-out) violates the first and is outside the model; Section 6.7 counts how often that shape appears in public code.

Determinism is not assumed by the theorems. Every mechanized result ranges over recorded histories with write values free; under sampling a value mismatch need not change the downstream decision, so the A_1 detector is a sound candidate screen whose over-reporting Section 6.1 measures (materiality), the stance of durable-execution engines [35].

Atomic commit is at the record level. External effects are outside the store and are governed separately: their order by A_6 and L_3, their release relative to a still-retractable basis by A_3 and L_2. At-most-once externalization is a hypothesis of this model; it is stated as the predicate D of Section 3.7, checked on every committed record (0 duplicate emissions among 12,900 externalized effects), and violated by one deployed resumption path we reproduce.

No replication; no one runtime. The memory is one logical store (split view under replication, A_4, is formalized in supplement § J); the model derives from no one runtime: LangGraph is driven through it live (Section 6.1), three others are placed informally (supplement § D).

3 THE ANOMALY CATALOG

Four anomaly predicates over histories are formalized in TLA$^+$ (`Anomalies.tla`); each has a TLC witness at $|A|{=}2$, $|\mathit{Cells}| \le 2$, $\mathit{MaxOps} \le 4$, reproduced by the artifact's `reproduce.sh`. Each predicate's classical counterpart and the regime feature that forces the port are named in its subsection and tabulated in supplement § R. The numbering (A_1, A_2, A_3, A_6) is kept from the public artifact (an earlier A_5 was subsumed by A_1); Section 3.6 defines, outside the catalog, the predicate for what the four leave behind.

3.1 A_1: stale generation

Definition 1 (Stale generation)**.** A history h exhibits *stale generation* when there exist $i \neq j$ with $h[i].\mathit{agent} \neq h[j].\mathit{agent}$ and $c \in h[i].\mathit{read_set} \cap h[j].\mathit{write_set}$ such that

$$h[i].\mathit{read_time} < h[j].\mathit{write_time} < h[i].\mathit{write_time}$$
$$\text{and} \quad h[i].\mathit{read_values}[c] \neq h[j].\mathit{write_values}[c].$$

Witness. TLC finds a five-state counterexample: a_2 reads $c_1 =$ NULL at $\mathit{read_time} = 0$; a_1 commits $c_1 = v_1$ at $\mathit{write_time} = 1$; a_2 commits at $\mathit{write_time} = 2$ with $\mathit{read_values}[c_1] =$ NULL.

Classical placement. The minimal witness is a single read–write antidependency from i to j, serializable in the order $i; j$, and none of Adya's proscribed phenomena [8]; TLC exhibits such a history (claim `A1OnlyOnNonSerializable`, violated as expected). A_1 is therefore not an isolation anomaly in the classical sense. Its *absence* is nevertheless a classical correctness class, and we state which one at the level at which the theorem holds.

Definition 2 (Commit-order read consistency)**.** A history is *commit-order read consistent* when every operation read, for every cell in its read set, exactly the value it would have read had it executed atomically at its own commit point: for all i and $c \in h[i].\mathit{read_set}$, $h[i].\mathit{read_values}[c]$ is the value of the latest write to c with $\mathit{write_time} < h[i].\mathit{write_time}$, or NULL if there is none.

Writes are installed in commit order in any case, so Definition 2 is value-level equivalence, on the reads, to the serial execution in commit order: the value-level analogue of Weikum and Vossen's commit-order-preserving serializability [3], not that conflict-based condition itself; under value-unique writes the two coincide on reads (view equivalence [26]).

Theorem 1 (A_1-freedom implies commit-order read consistency)**.** *Let h satisfy the store axiom, read-before-commit and per-agent non-overlap. If h has no A_1 witness, then h is commit-order read consistent.*

The proof (`CommitOrder.tla`; TLAPS, 60 obligations, no omitted step) is Kung and Robinson's backward-validation argument run against our definitions: if some operation's read of c disagrees with the latest write to c before its commit, that write lies inside the operation's window, was made by another agent (one agent's operations do not overlap), and wrote a different value—an A_1 witness. Fifteen TLC verdicts (`check_commitorder.sh`) fix what the statement says, and all reproduce: over all 35,659 histories of length at most three the implication holds, is not vacuous, and fails when either droppable hypothesis is removed; the relational definitions equal `Memory.tla`'s and a literal serial replay; the three hypotheses are invariants of the operational model over 2,961,434 reachable states; and the converse fails in exactly one shape, a value overwritten and then restored inside the window. The predicates nest strictly:

$$\neg\mathrm{CORC} \implies A_1 \implies \text{validation fails.}$$

Reach. The theorem is a conditional guarantee over faithful histories that satisfy its three hypotheses; it says nothing about serializability of an application, about effects, about reads the

instrumentation did not declare, or about a runtime that violates per-agent non-overlap—LangGraph's `Send` fan-out, several instances under one agent name, is outside the model (Section 2). On the committed data all three hypotheses hold where they can be checked (`hypothesis_gates.py`, failing on the first violation): 0 same-agent overlapping windows and 0 reads not before their commit in 84,282 operation records over 22,188 traces (ledger registration records are not operations), and 0 store-axiom violations over 44,545 declared reads in the 19,666 traces that record their store; the 4 campaigns converted from transcripts and tool logs are outside the axiom's reach and supply no rate of this paper's tables: the 16 campaigns behind them are store-instrumented (0 converted; gated), so every A_1 verdict here is taken on records the hypotheses were checked on. The converse shape is a false positive of L_1, priced in discarded generations; on the live records it occurs in 0 of 1,299 A_1 witness pairs, and cannot on these workloads, whose revisions are monotone. Two corollaries are mechanized with the theorem: the classical lost update (P4 of Berenson et al. [7]) is an A_1 witness whose stale reader also writes the cell it read, so L_1 excludes it; a *blind* overwrite—a write by an operation that never read the cell—is outside isolation theory and the catalog, and is the kind of the one in-the-wild bug we reproduce (Section 6.7) and of a revision landing *after* the reader's commit (Section 6.1).

3.2 A_2: phantom tool

Definition 3 (Phantom tool)**.** h exhibits a *phantom tool* when for some i,

$$h[i].\mathit{planned_tool} \neq \text{NULL},$$
$$h[i].\mathit{planned_tool} \in h[i].\mathit{read_registry},$$
$$h[i].\mathit{planned_tool} \notin h[i].\mathit{write_registry}.$$

An agent selects a tool from the registry it read; the registry mutates during generation; the commit emits a call to a capability that no longer exists. TLC's witness has four states. Definition 3 is stated over identifiers; the Verus restatement of Section 5.4 maps identifiers to signatures and fires additionally when a tool is re-signed under its old name. The two coincide under the encoding the executable dispatcher uses (re-signing removes the old pair and inserts a fresh one); the scorer and the L_4 dispatcher decide the pair form, and Table 1 records the correspondence as an encoding, not a proof. A hallucinated call to a tool never in *read_registry* is not A_2 and not in the catalog.

3.3 A_3: causal cascade

When an operation is retracted in a compensating (saga-style) runtime, any operation that read a value it produced is grounded on a basis no surviving commit supports. The classical remedy, cascading abort, flags the dependent aborted; if the dependent's tool effects have already been issued, the flag changes nothing that reached the world, so the cataloged predicate is stated over externalization.

Definition 4 (Causal cascade)**.** Let each record carry *aborted* and *externalized* flags and a predecessor set *preds* (the operations whose committed writes it observed, transitively closed). h exhibits a *causal cascade* when for some j, p:

$$h[j].\mathit{externalized} \wedge p \in h[j].\mathit{preds} \wedge h[p].\mathit{aborted}.$$

The surviving-dependent form, $\neg h[j].\mathit{aborted} \wedge p \in h[j].\mathit{preds} \wedge h[p].\mathit{aborted}$, is kept beside it as CASCADEUNPROPAGATED; a cascading abort falsifies that form by flagging j after j's effects are out, and Section 6.4 measures the separation on runnable arms.

Prevention is output commit, and output commit has a residue. Under Definition 4 a runtime must hold an operation's effects (output commit, in the sense of optimistic recovery [18]) until every operation in its closure has released its own, and must refuse to retract an operation whose effects are out. The first clause is prevention; the second keeps the predicate false by refusing to set the flag it reads, which we do not count as prevention—a refused retraction is a basis someone wished to withdraw after release, the ordinary rule that a committed transaction cannot abort—and Section 6.4 counts its residue beside the zero.

What the detector decides. A trace without provenance fields cannot decide Definition 4. What it can decide is an *unsupported read*—a committed read of a value that no surviving committed write produced at or before the read. The verified `detect_a3` decides exactly that screen, and its relation to Definition 4 is five mechanized facts (supplement § G): it is not sound in general (a value collision hides the aborted writer), it is sound under value-unique writes, the hypothesis black-box isolation checkers also need [32], [33], it holds along chains, it is not complete, and it is blind to relabeled effects. We therefore claim Definition 4 only where the hypothesis holds, and check it: value-unique writes hold by construction in the synthetic cascades and in all 600 live replay sessions (0 collisions), the two data sets the detector scores; they fail in 238 cells of the pilot, refusal-study, occurrence-study and one exec-ledger campaign, on none of which A_3 is scored. A recorder that stamps each written value with its writer's operation identifier makes the hypothesis a system invariant.

3.4 A_6: tool-effect reordering

Each record additionally carries an issuance order *io* and a commit order *co* over its external effects; *write_set* and *write_values* derive from *co*.

Definition 5 (Tool-effect reordering)**.** h exhibits *tool-effect reordering* when for some i,

$$|h[i].\mathit{io}| \geq 2, \quad h[i].\mathit{co} \neq h[i].\mathit{io},$$
$$h[i].\mathit{co} \text{ is a permutation of } h[i].\mathit{io}.$$

The minimal witness (three states, one agent, two cells) is benign over store cells, where writes to distinct cells commute; the predicate names an anomaly only when each element of *io* is an external effect whose order an observer can see. It over-reports on commuting effects and, requiring a permutation, is silent on dropped and duplicated effects (Section 3.7 states the duplicated case; Section 6.7 reproduces it).

3.5 Snapshot insufficiency

L_1 is not a snapshot guarantee: "reads reflect memory at read time" is the store axiom of Theorem 1, holds on every reachable log, and TLC exhibits a history satisfying it on which A_1 fires (`MC_A1_struct`; supplement § B); LangGraph's superstep pins a consistent snapshot per node and has not thereby obtained L_1 (Section 6.1).

3.6 What the catalog leaves: the residue predicate

Every predicate above is decided at or before an operation's commit, and a discipline that excludes all four still leaves a harm the live

study measures in 1–65% of sessions per family and arm on the API families (Section 6.1): the external effect a commit released is irreversible, while the state it was derived from is ordinary revisable state that any later operation may overwrite. This is not a concurrency anomaly. It fires in correct sequential executions—book against the date known at commit, then be contradicted by a legitimate revision—and classical isolation has no name for it because a committed transaction's inputs are final by definition; the nearest notions are view maintenance [65] with the view outside the system, and the saga's compensation [17]. We define it because an evaluation scored on outcomes needs a predicate for the outcome's other half, and keep it outside the catalog and the lattice.

Each operation record carries a *basis*, the (cell, value) pairs its external effect was derived from, and an *obligation* flag. The basis is recovered by *argument provenance*: a read value that appears verbatim among the effect's arguments is part of its basis, and nothing is declared by the agent. This is why-provenance [64] restricted to copying, with that tradition's known limit: a value that reaches the effect through a computation leaves no basis, so the extractor under-reaches and never over-reports. In the model the basis is a nondeterministic subset of the read values, so every extractor is covered.

Definition 6 (Residue predicate)**.** h exhibits a *superseded-basis effect* (R; `NoA7` in the artifact) when for some i, $h[i].externalized$, $\neg h[i].obligation$, and for some $(c,v) \in h[i].basis$ the *latest* write to c committed after $h[i].write_time$ wrote a value other than v:

$$\begin{aligned}\exists j:\ & c \in h[j].write_set \ \wedge\ h[j].write_time > h[i].write_time \\ & \wedge\ h[j].write_values[c] \neq v \\ \wedge\ \forall k:\ & (c \in h[k].write_set \wedge h[k].write_time > h[i].write_time) \\ & \Rightarrow h[k].write_time \leq h[j].write_time.\end{aligned}$$

On a workload that revises a cell once, A_1 and R partition "an effect is out on a basis the state no longer holds" by whether the superseding write committed inside the operation's window or after its commit (with two revisions one effect can fire both); a value overwritten and then restored fires neither, and R carries no agent conjunct because the obligation is owed whoever supersedes. On the live data the partition is exact (Table 2): $A_1 + R = 100$ in every unguarded cell, and under L_1 the residue is entirely R. The identity is not evidence for the extractor: on a workload that copies the date verbatim into the booking it holds whenever the extractor recovers the basis at all, and it recovers 9,000/9,000 because the harness writes the read value into the effect's argument; we keep it as a gate (`a7_rescore --check`) and state the extractor's blind spot in Section 8. TLC's witness has five states and is not an A_1 witness: an operation reads $c_1 = \text{NULL}$ and commits an effect with basis $\{(c_1, \text{NULL})\}$, released at commit; a later operation commits $c_1 = v_1$.

What remains after release (a remark, not a result). Under the *open-world hypothesis*—the store accepts, at any time after an effect e with basis B is released, a write to a cell of B by an operation that does not depend on e—two things are immediate and fix what a discipline can promise. (i) Some execution supersedes B while e is out, because the write is admitted after a release that cannot be recalled. (ii) If obligations are never cleared, $\neg R$ holds in every reachable state iff, in every execution, e's obligation is set in the same step as, or before, every step that completes the other conjuncts of Definition 6 (the superseding commit for a released effect; the release for an effect superseded while held), because Definition 6 is "superseded and not owed" and a monotone flag set by the first such step stays set. So a discipline can promise not a current basis but an obligation registered atomically with the step that completes the witness, which the ledger model of Section 5.5 does at both steps (the executable ledger registers inside the store lock with the write and clears a discharged obligation; never-cleared obligations are the model's). Dropping the hypothesis is not free: freezing the cells of B until the effect is acknowledged is a lock held past commit, priced by the pessimistic column of Table 5; output commit (Section 3.3) holds effects only until their *causal* basis is externalized, a finite condition. What a registered obligation buys in the world is a property of the compensator and is measured, not proved (Section 6.2).

3.7 What the model assumes about effects: at-most-once, as a predicate

The model externalizes an operation's effects once, at commit. LangGraph's `interrupt` resumes a node by re-executing it from its start, so an irreversible effect issued *before* the gate is emitted again after it, with different arguments when the shared value was revised during the gate (Section 6.7; 2 emissions of one pre-gate effect in the probe). Rather than record this as a limitation we state the hypothesis as a predicate over the effect stream: D holds when two emissions carry the same node's effect, $D^{\neq}$ when they also differ in their arguments. A five-action TLA$^+$ module (`Duplication.tla`: issue, gate, resume-from-start, finish, revise) reaches D in 5 steps and $D^{\neq}$ in 6; with an idempotency key per (node, effect) TLC finds neither over 1,714 states exhaustively. D stays outside the catalog and the lattice like R: its carrier, the effect stream, is disjoint from the four disciplines', which neither create nor remove it. On the committed operation records 0 of 12,900 externalized effects were emitted twice (`hypothesis_gates.py`, H5); no campaign of those records drives LangGraph's `interrupt`, and the probe's two emissions are in its own receipt, outside that count.

4 The Lattice and One Chain Through It

A consistency model is a predicate over histories, characterized by the set of anomalies it excludes [8]. With $\mathscr{A} = \{A_1, A_2, A_3, A_6\}$ the design space is the Boolean lattice $\langle 2^{\mathscr{A}}, \subseteq \rangle$: sixteen points, bottom $\emptyset$ admitting everything, top $\mathscr{A}$ admitting nothing (drawn in supplement § L). We name five points on one maximal chain,

$$L_0 \equiv \emptyset,\ L_1 \equiv \{A_1\},\ L_2 \equiv \{A_1, A_3\},\ L_3 \equiv \{A_1, A_3, A_6\},\ L_4 \equiv \mathscr{A},$$

chosen on operational grounds (A_1 first because it is the basic read–generate–commit failure; A_2 last because registry stability across the operation lifetime is the costliest single requirement); the other twenty-three linearizations are equally valid and the lattice, not the chain, is primary. The residue predicate R is not a generator: it is not an anomaly, and its answer, the basis ledger ℓ of Section 5.5, composes with any point.

4.1 What is and is not a result

The chain's inclusions are definitional and its strictness needs one witness per step; neither is a result. What requires work is *realizability*: a runtime configuration that excludes exactly each point's anomalies, and for the five named points a verified realizing runtime—established here by bounded model checking at two agents, two cells and two operations (three for A_3 and for the

supersession class of A_1), one value for the witness half and two for the safety half, and claimed at exactly those bounds, which exercise every predicate's minimal witness and nothing longer; the unbounded statement exists only for the top point, as the Verus product of Section 5.6.

4.2 All sixteen points, mechanically; the ledger composes

`Guarded.tla` extends the runtime with four commit-time disciplines, each enabled by a Boolean constant and each acting on its own carrier: G_1, read-set validation (an operation whose read set was overwritten after its read may not commit); G_2, registry validation (a planned tool no longer in the registry may not be dispatched); G_3, causal tracking with output commit (an operation records the closure of committed writers whose values it read, may not commit while any is aborted, externalizes only once its closure has externalized, and once externalized is irrevocable; a retraction cascades to every unexternalized dependent); G_6, commit-order sequencing (effects externalize in issuance order). The carriers are disjoint—read set and write times; registry and planned tool; closure and abort flags; issuance and commit orders—so enabling one guard neither creates nor removes another anomaly, and the matrix confirms it on every combination.

For each of the sixteen guard sets S and each anomaly a, TLC checks $R(S) \models \neg a$: for $a \in S$ the invariant must hold exhaustively, for $a \notin S$ it must be violated. All 64 runs match (`lattice16.sh`; $|A|$=2, $|Cells|$=2, $|Values|$=1, $MaxOps$=2, symmetry on agents and cells; the 32 safety runs are exhaustive over 380,920–2,015,279 distinct states). Because CAUSALCASCADE ranges over flags and predecessor sets and never over cells, the sixteen NOA3 points are rechecked at $MaxOps$=3 with one cell, where a transitive cascade first becomes reachable: 16/16 match. A single-value domain makes Definition 1's value conjunct vacuous on non-null reads; the safety half re-run at $|Values|$=2 on the current model stays safe (all 32 safety runs SAFE, 2.4–17.3M states), and a targeted check shows the non-null-read witness class—the live study's supersessions—needs both a second value *and* a third operation (supplement § L), so at $MaxOps$=2 the NOA1 safety verdicts exclude only the cold-start class; the sixteen NOA1 points are therefore rechecked at $MaxOps$=3, $|Values|$=2 and one cell, where that class is reachable (`lattice16_a1_maxops3.sh`; supplement § L): all 16 runs match (the 8 G_1 points safe over 3.0–10.2M states; the supersession class, a non-NULL stale read, reached without G_1 in 18,242 states and excluded with it); the two-cell universe at that depth did not close in 25 minutes (72M states) and the verdict covers one cell. Every point is thus inhabited by a runtime that excludes exactly its avoidance set (soundness only), at these bounds; the Verus product of the L_2, L_3 and L_4 models (Section 5.6) mechanizes the top point without a bound.

The ledger composes. `Guarded5.tla` adds a fifth constant G_7, the basis ledger of Section 5.5, which refuses nothing and writes only the basis and obligation fields. `lattice32.sh` runs all 160 combinations of the five constants against the four anomalies and R at the same bounds. The 64 anomaly cells with G_7 on return the verdicts of `lattice16.sh` one for one, so the ledger disturbs no point of the lattice; R is reached without G_7 in a five-state trace at every point and excluded with G_7 at every point (exhaustive; 5,213,012 distinct states at the bottom point, 9,688,601 with output commit); no run mismatches its expectation.

5 MECHANIZATION

Three tools carry the development, at assurance levels Table 1 labels claim by claim. TLC [78] checks the witnesses, the lattice and ledger matrices and the fifteen Theorem 1 verdicts (bounded, exhaustive within bounds). TLAPS [79] proves Theorem 1 (60 obligations), a coherence check of the chain definitions (`Hierarchy.tla`, 15) and a same-agent exclusion lemma (`A1LowerBound.tla`, 28); only the first is a substantive result. Verus [21] carries the substantive mechanization: 371 obligations (392 counting four helper files), zero errors, no `assume` or `admit` in any proof body, regenerated by `verus_count.sh` against a pinned build. Of these, 236 bear a deployed or measured artifact's safety claim and 135 are scaffolding (superseded refinements, the A_4 model, the model-only Verus mirror of `Guarded.tla`); supplement § S inventories every file.

5.1 Verified detectors, and one transcription gap

The four detectors are verified sound *and* complete against Verus `spec` restatements of the TLA$^+$ predicates (24 obligations). `detect_a1` is a two-sided equivalence whose completeness half strengthens each of three nested loop invariants to carry "no witness in the prefix searched." `detect_a2` and `detect_a6` are single-record equivalences; `detect_a3` is exact for the flat-trace residue of Section 3.3.

The restatement is hand-written and checked by inspection, and inspection found a gap we state rather than hide: the Verus `OpRecord` carries no agent field, so its A_1 predicate omits the different-agent conjunct of Definition 1, and it decides "no witness" when a read value is absent, where the TLA$^+$ compares against NULL. Neither matters on the histories the model admits.

Lemma 1 (Same-agent exclusion)**.** *Under per-agent non-overlap and read-before-commit, if* $c \in h[i].read_set \cap h[k].write_set$ *and* $h[i].read_time < h[k].write_time < h[i].write_time$, *then* $h[i].agent \neq h[k].agent$.

The lemma is step $\langle 2 \rangle 2$ of the TLAPS proof of Theorem 1 and the content of `A1LowerBound.tla`. It makes the Verus predicate and Definition 1 coincide on every history satisfying the model's invariants; on a history that violates non-overlap the Verus predicate is strictly stronger—it over-reports, the safe direction for a detector. The instrumentation records a value for every read cell, so the absent-value case does not arise: across all 84,282 committed operation records there is no read cell without a value and 0 same-agent overlapping windows (`hypothesis_gates.py --check`).

Which bytes score A_1. Agreement of three transcriptions on committed data is testing, not equivalence, and the earlier version of this paper reported tables scored by a hand-written port with such a gate. The scorer now takes its A_1 verdict from `detect_a1` itself: the verified exec-mode file (`lib_detect_a1_exec.rs`, 9 obligations, sound and complete against `is_a1_witness`, the gated restatement's conjuncts over (cell, value) pairs; agreement checked by inspection and on every cell by the gate) is compiled by plain cargo into the runtime crate byte-identical to the verified source, behind a 60-line wrapper that interns cell names and values injectively, drops a duplicated declared cell (precondition `wf_hist`) and encodes a read cell without a value as a sentinel no record can carry—the one unverified piece between the verified bytes and a string-valued record, trust-base item (xii). The port is kept beside it and the binary refuses a trace on which the two

TABLE 1
Every claim at its assurance level. *Model*: checked by TLC within bounds; *proof*: TLAPS or Verus, no `assume`/`admit`; *bytes*: the verified code is the code measured; *port*: unverified code gated against a verified artifact on the committed data; *open*: stated, not established. Correspondences name the hypothesis under which two formalizations agree and the gate (`harness/macaudit/`) that checks it.

Claim	Definition	Proof / check	Executed by	Correspondence, hypothesis, gate; status
Anomaly witnesses, lattice realizability	TLA$^+$	TLC, $\|A\|$=2, $\leq$ 2 cells, $\leq$ 2 values, MaxOps $\leq$ 4 (witnesses), $\leq$ 3 (matrix)	—	**model**: bounded; 64+16+32 two-value safety runs
Theorem 1	TLA$^+$	TLAPS 60	—	**proof**; its three hypotheses are model invariants (TLC) and hold on the committed records (`hypothesis_gates` H0, H1, H7: 0 violations; Section 3.1)
`detect_a1` sound & complete	Verus spec	Verus	scorer: verified bytes behind an interning wrapper	**proof** against the restatement; restatement = Definition 1 under Lemma 1; **bytes**, gated against the port on every cell; the gate's clean receipt **[pending: step 14 rerun]**
`detect_a2`	Verus spec (id, signature)	Verus	L_4 dispatcher (bytes); scorer (port)	**proof**; **port** in the scorer; = Definition 3 under the re-sign-as-replace encoding, **stated, not proved**
`detect_a3`	Verus spec (unsupported read)	Verus	scorer (port)	**proof** for the screen, **port** in the scorer; = Definition 4 under value-unique writes (gate H2: 600 sessions, 0 collisions)
`detect_a6`	Verus spec	Verus	scorer (port)	**proof** of the detector, **port** in the scorer; single-record equivalence to Definition 5
L_1 stores window-free	Verus	Verus (lock predicate)	pilot (bytes); live *verified* arm (bytes via bridge); *validated* arm (application layer)	**bytes** in the pilot and the verified arm, **port** in the validated arm (check-then-put, not atomic; a slip would be an A_1 witness, 0 in 3,900 sessions); refinement to the abstract machine **open**
L_2 output commit	Verus	Verus	`l2_exec` (bytes)	**bytes**; sequential object, caller serializes
L_3 sequencing	Verus	Verus	`l3_exec` (bytes, via `l3_server`)	**bytes**; bridge trusted (xiii)
L_4 validation at dispatch	Verus	Verus	`l4_exec` (bytes)	**bytes**
R on the live records	Definition 6	—	`a7_rescore` (Python)	**port**, unverified; independent of the outcome column; extractor coverage 9,000/9,000 (coverage, not recall)
Basis ledger excludes R	TLA$^+$ G_7; Verus model	TLC 160 runs; Verus verified (7 obligations)	probe 303/304 ledger arms (Python, over L_1); bookkeeping in `l5_exec` (bytes, via `l5_server`)	**model** and **proof**; the executable ledger is a **port**, its bookkeeping **bytes** (10 obligations, 0 errors); hook, interning and compensator trusted; in a world its worth is the compensator's, measured (Section 6.2)
D (at-most-once) excluded by an idempotency key	TLA$^+$	TLC: witness, exhaustive with the key	—	**model**: bounded (1,714 states); on the records 0/12,900 duplicates (`hypothesis_gates`, H5)

disagree. Every published A_1 cell was rescored through the verified bytes (`verified_scorer_gate.py`: 144 cells, 14,400 traces, each verified count equal to an independent Definition-1 count), but the gate exited non-zero on 18 per-cell `refusals.jsonl` sidecars of the refusal study that it handed the analyser as op-records (a reader defect, like the one its first pass caught; supplement § W.5), so its clean receipt is **[pending: runner step 14 rerun]** and until then the tables also stand on the gated port.

5.2 L_1: two concurrent stores whose lock is verified, not assumed

Three abstract state machines—pessimistic locking, serializable snapshot isolation (SSI), and default snapshot isolation with its no-write bypass—are proved A_1-free under sequential interleaving (40 obligations; default-SI conditionally, on workloads with no read-only commit, which is the residual Section 6.3 measures). The deployed stores do not rest on these: each is an exec-mode store whose only access path is `vstd::rwlock::RwLock`, the reader–writer lock implemented and verified inside the Verus standard library, carrying the store's invariant as the lock predicate (`lib_si_concurrent.rs`, 18; `lib_pess_concurrent.rs`, 31; no axiom, no `external_body`). The invariant entails the absence of any stale-generation window in the store's own record stream, under any interleaving of threads. Theorem L1j (`lib_si_projection.rs`, 3) transports that guarantee to the detector's history for *faithful* emitters—one `OpRecord` per store record, cells a subset of the record's. The default-SI `tick` path is the standing counterexample to faithfulness and is exactly the 5%/8% residual of Section 6.3. Four earlier refinements (86 obligations, two axioms) relate the abstract machines to representations shipped *before* the verified-lock rebuild and bear no deployed claim; refining the two concurrent stores to the abstract machines is the open proof step (a bounded simulation over 506,880 transitions finds no counterexample; supplement § H).

5.3 L_2: output commit

The L_2 model (`lib_l2_safety.rs`, 33) carries per transaction a predecessor set recorded as a causal closure at read time. A commit is valid iff the read set is fresh against the committed store and every predecessor is committed and unaborted; an abort cascades to every transaction whose closure holds the aborter; effects leave by a separate `externalize` step enabled only once the closure has externalized; an externalized transaction cannot abort. Theorem L_2h: no state satisfying the inductive invariant *inv_output_commit* contains a witness of Definition 4 (`lemma_l2_reachable_no_a3`), under arbitrary interleaving of the six steps; the closure invariant makes the one-level cascade provably sufficient for the transitive case. Theorem L_2i: every committed read has a surviving producer of exactly its value, committed no later than the read. Theorem L_2j (`lib_l2_projection.rs`, 3): a faithful emission of an L_2-supported state contains no residue witness. The exec runtime (`lib_l2_exec.rs`, 73 end to end, 40 net of the re-included model) refines the model per transition with zero added axioms; every mutating entry point checks its own guard, so the guarantee survives proof erasure under plain `cargo`, and the file the reference runtime compiles is byte-identical.

5.4 L_3: sequencing, and L_4: validation at dispatch

Theorem L_3g (`lib_l3_sequencer.rs`, 5): a commit-order sequencer that externalizes effect k once effects $0..k$ have completed yields an externalization time monotone in issuance index for *every* completion schedule, hence no A_6, without forbidding

concurrency; a reversed schedule exhibits A_6 unsequenced and none sequenced. The saga model (`lib_l3_safety.rs`, 6) is the strict-serialization special case and composes with L_2 on disjoint carriers. The exec sequencer (`lib_l3_exec.rs`, 7) is verified and compiled byte-identical into the runtime crate; its measurement (Section 6.5) runs those bytes behind a 70-line JSON bridge that releases real effects in the order they decide (trust-base item xiii).

Resolving a planned tool against a registry snapshot does not prevent A_2: a snapshot changes what an operation resolves, not the tool its call reaches (Theorem L_4d, a concrete execution). The L_4 model (`lib_l4_safety.rs`, 9) reads A_2 from a *dispatch record* and validates at dispatch—place the call only if the planned tool is still live with the pinned signature—so registry churn cannot create A_2, a property of the dispatch. The exec dispatcher (`lib_l4_exec.rs`, 11) refuses a revoked or re-signed tool and dispatches an untouched one (Section 6.5 measures those bytes). Validation and dispatch are one atomic step of a registry the runtime owns; a remote tool server (MCP [22]) offers no such atomicity, and the check-then-call gap there is narrowed, not closed.

5.5 The basis ledger

The ledger ℓ answers R and composes with any discipline; the live study runs it over the application-layer L_1 (Table 2; the arm over the verified store is the *ledger-verified* row of the replicated run). Its model (`lib_l5_ledger.rs`, 7 obligations, 0 errors) carries the store's committed values and, per effect, its basis map, an externalized flag and an obligation flag. Three steps: *register* commits an effect with its basis (released at once, or held under output commit; if released at once on a basis the store already disagrees with, the obligation is set at commit); *write* installs a value and marks, in the same step, every externalized effect whose basis holds that cell with a different value; *externalize* releases a held effect and marks it if its basis is stale by then. The invariant *inv_l5*—every externalized effect with a stale basis carries an obligation—is preserved by each step (the write step is the load-bearing case: the cell that became stale is the cell the step marked, and any other stale cell was stale before), and the ledger theorem: no state satisfying *inv_l5* contains a witness of Definition 6. The model's witness is the state form—an externalized, unowed effect whose basis the store now disagrees with—which contains Definition 6's (a latest post-commit writer that differs implies a current value that differs), so excluding it excludes the definition's witness and an effect released on a basis already stale at commit. Non-vacuity in both directions: the unguarded write reaches R in two steps from an effect derived from an unwritten cell, and under the ledger the same write registers the obligation while a write that restores the value leaves no witness. The ledger refuses nothing; refusing to release a stale-basis effect, or holding it for a deadline, is a policy the runtime layers on top. The invariant speaks of records: a compensator that reports a discharge while a stale confirmation stays active satisfies it vacuously, which is what Section 6.2's world check found. The model is verified (7 obligations) under the pinned build; its obligations count as scaffolding (135 with it) because the executable ledger of the live study is harness code, not these bytes; it registers the obligation inside the store lock, in the critical section of the superseding write, and runs the compensation in that writer's thread after the lock is released. An exec ledger (`lib_l5_exec.rs`, supplement § S) carries the bookkeeping alone—which effects a write obliges (total: no superseded basis without an obligation), what a compensator's verdict does, what is open—behind a JSON bridge; the store hook, the interning and the compensator stay in the trust base. The ledger arm of Section 6.2 rerun with its bookkeeping in those bytes (proofs erased by cargo; server `f141b709aaaa0040`; three families): world wrong 6–12, none 6–10, agreement 300/300, the bytes' open set equal to the shadow's in every session—the same bookkeeping, the residue the world's. The file is verified (10 obligations, 0 errors) and byte-identical in the runtime.

5.6 Composition

`lib_consistency_lattice.rs` takes the product of the L_2, L_3 and L_4 states over their disjoint carriers and proves that the conjunction of the three invariants excludes A_1 (image), A_3, A_6 and A_2 jointly, with three non-interference lemmas (53 obligations, 48 of them the re-verified components). The ledger's carrier (basis, obligation) is disjoint from all three; adding it to the product is the same construction, not in this version.

5.7 Trust base

"Zero `assume`, zero `admit`" is a statement about proof bodies. What is trusted, in full: (i) Verus, its SMT backend and `vstd`, including `group_hash_axioms` for the `HashMap`-backed exec states; (ii) two axioms in the four superseded refinements (string injectivity, null sentinel) and (iii) three `external_body` stubs for a `parking_lot::Mutex`, neither bearing a deployed claim since the verified-lock rebuild; (iv) the hand restatement of the TLA$^+$ predicates in Verus, with the gap of Section 5.1 stated and gated; (v) the hand-written ports that score A_2, A_3 and A_6 (A_1 is the verified detector, item xii), gated against the verified predicates on every committed cell; (vi) for L_3, the inspection-level correspondence between the exec artifact and the measured twin; (vii) the `pub` fields of the exec runtimes, which a caller can write around the API; (viii) the marshalling that flattens the version map into the verified validation gate, and the interning bijection and abort counter outside the verified store; (ix) the Python and Rust instrumentation that produces every `OpRecord`, including the read sets it declares—the largest item: a detector is as good as the read set it is handed, and S-Bus [38], our own read-set reconstruction from HTTP traffic, finds 74% of reads on a SWE-bench workload invisible to that surface; at the store boundary, the keys a node read between its first read and its commit equal its declared read set (gate H6: 0 undeclared store reads over 8,400 records of probes 303 and 304), while a value that reaches the model through prompt context neither instrument sees. (x) the provenance correspondence between the two formalizations of A_3, which agree only under value-unique writes (checked on every scored trace); (xi) in the live study's verified arm, the 100-line JSON bridge and the harness's interning of cells and values; (xii) the 60-line interning wrapper between the verified A_1 detector and a string-valued record (Section 5.1); (xiii) the 70-line bridge between the verified L_3 sequencer and the tool executor (Section 6.5); (xiv) for the exec ledger of Section 5.5, the bridge with its Python shadow, the store hook that calls it inside the lock, and the compensator. No safety-bearing proof invokes a writes-as-function-of-reads assumption, and the development carries no probabilistic obligation.

6 Evaluation

Three questions. **RQ1** For each discipline, what does it prevent, and what harm remains once it has? **RQ2** What does prevention

cost, per unit of work the discipline actually delivered? **RQ3** Where do the structural preconditions of the anomalies occur in deployed code? A verified arm's zero is a theorem, so every zero below is reported beside a count that reads no runtime flag.

Provenance. Every table cell maps to a committed trace directory and a scorer in the artifact; `reproduce.sh` rescores all of them offline under an implementation of Definition 1 independent of the verified detector and the Rust port, and fails on any disagreement. Model-driven cells are one run each at the stated n unless replicated; a regeneration reproduces the protocol and the intervals, not the sessions. Intervals are exact Clopper–Pearson unless stated.

6.1 RQ1 on a live runtime: what L_1 removes and what it leaves

The scenario of Section 1.1 runs on a released LangGraph runtime (`langgraph` 1.2.0) with live models. Two compiled graphs share one `BaseStore`, the framework's cross-thread memory, subclassed only to stamp a logical clock at `get`/`put` under one lock, the one instrumentation (`BaseStore` offers no version or compare-and-set, so the framework as shipped cannot validate a read set). A flight agent reads the trip date, drafts a reservation over a model call and puts the booking; a date agent, started by the harness 150 ms after the flight agent's read, reads the date, asks its model for a one-week revision and puts it. Three modes: *vanilla*; *validated*, the L_1 discipline at the application layer (re-read before `put`, discard the generation and regenerate on change); and *sequential*, a control in which the date agent runs after the flight agent commits. 100 sessions per cell on three model families. The booking's date is the value the flight agent read; the models draft the texts and set the latencies, and model output is scored only in the materiality study below. Each session records the A_1 verdict, the regenerations, and the booking's and final trip dates (Table 2).

The window is open and the framework does not close it. On both API models the runtime committed the stale booking in every session in which the revision landed inside the window (60% and 91%), the detector fired with the exact temporal witness of Definition 1, and nothing in the framework blocked, validated or reported any of them. A value mismatch may be lexical: in the live study the stale date reached the booking text in 150 of 152 firings (September run); in a controlled triage task, varying only the shared value an agent read changed its decision in $150/200$ flagged mismatches (75.0% $[68.4, 80.8]$), the paper's only value-staleness estimate; on the pilot's firings materiality is $43/43$ under an LLM judge (supplement § N). Behind a request-serializing local server the open-weights family's revision landed after the commit in $99/100$ sessions (the one firing had the longest pinned interval, 9.06 s): a property of the serving configuration, not of the model.

What L_1 removes, and what it does not. Read-set validation removed every firing ($0/300$) at one discarded generation per realized conflict (66 and 93 per 100 sessions, never two in one session). It did not make the booking agree with the final trip date: in 34 and 7 of 100 sessions the revision landed after the commit—the record is current at its commit point exactly as Theorem 1 says, and the booking is stale against a state that did not yet exist: a post-commit supersession, the blind overwrite of Section 3.1, outside every isolation discipline, reached only by compensation or by holding the effect until its basis is final. The *sequential* control, clean under every commit-time predicate, delivers the wrong booking in $100/100$ sessions for the same reason; the predicate alone would have credited L_1 with a correctness it does not provide.

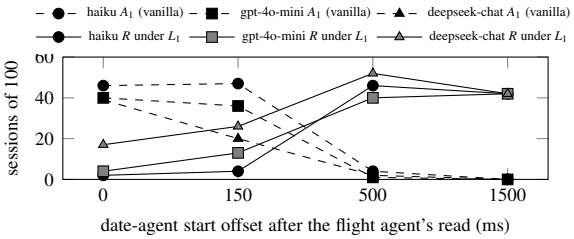


Fig. 2. Probe 303, offset sweep: the live A_1 rate without a discipline (dashed) and the residue R that read-set validation leaves (solid; equal to booking $\neq$ final, since A_1 is 0 there) as the date agent's start is delayed; revision probability 0.5, $n = 100$ per point. As the window closes the same revisions land after the commit: A_1 falls to 0 while R rises to the revision rate.

R decides the residue on the records. The R column of Table 2 is decided on the emitted records alone: the basis of each booking is the date value in its arguments, and R fires in exactly the sessions whose booking disagrees with the final date and A_1 does not ($A_1 + R = 100$ in every unguarded cell; under L_1 the R column equals the residue; `a7_rescore.py --check`), two independent computations whose agreement rests on the structured date argument (Section 3.6). Two further arms run on three API families (haiku, gpt-4o-mini and deepseek-chat through its OpenAI-compatible endpoint). The application-layer arm's validating re-read and its `put` are two store calls, so it is check-then-act, not an atomic commit: a revision landing between them would commit stale, and would be an A_1 witness on the record, whose read time is the validating read's; none did in the 3,900 sessions of this study that ran over it (2,700 validated, 1,200 ledger), which bounds the window without closing it. In the *verified* arm the L_1 decision is taken inside the verified store's lock through the bridge of Section 5.7: A_1 fires in $0/300$ sessions and the residue is 2, 51 and 52 of 100 against 12, 51 and 63 for the application-layer arm of the same run. In the *ledger* arm a superseding put registers the obligation and a compensator issues a replacement booking: $0/100$ wrong on all three families (5, 45 and 53 obligations, each discharged by one compensation generation), at the price of L_1's discarded generations plus one per obligation and a stall of the superseding writer, in whose put the compensation runs (Section 6.2).

What a difference between arms can mean, and what the replication says. The four arms of Table 2's October run were sequential blocks of 100 sessions, and the L_1 residue is the share of revisions that landed after the commit—a race between two provider latencies plus the fixed offset—so a latency drift between blocks moves it without any change of discipline: the two L_1 arms on haiku, one discipline decided in two places, differ at 12 vs. 2 of 100 (Fisher exact $p = 0.010$). The attributable design interleaves the arms per session in a seeded order, with fresh seeds per run; Table 3 reports 3 such runs of five arms per family (the fifth, *ledger-verified*, is the ledger over the verified store). The arms ran the same session back to back, so the comparison is paired: the validated arm alone left the residue in 7 vs. 2 sessions of 300 on haiku (McNemar exact $p = 0.180$; difference $+1.7$ points, Newcombe 95% $[-0.5, +4.1]$), 72 vs. 67 on gpt-4o-mini ($p = 0.735$; $+1.7$ $[-6.0, +9.3]$) and 58 vs. 67 on deepseek-chat ($p = 0.474$; -3.0 $[-10.2, +4.3]$): the replication detects no arm difference and bounds one at four points on haiku, about ten elsewhere; the sequential-run haiku difference did not replicate. A_1 is 0 in the 900 October-sequential and 3,600 interleaved sessions run under L_1 at this offset. The run, not the

TABLE 2
Live cross-agent stale generation on a released LangGraph runtime (store instrumented only to stamp a clock), n=100 per cell at a 150 ms race offset; the date agent revises in every session. A_1: Definition 1 on the emitted records. R: Definition 6 by argument provenance. *Booking ≠ final*: the harm, read off the store. *Regen.*: discarded generations. *Validated*: L_1 at the application layer; *verified*: the commit decision taken by the verified L_1 store through a JSON bridge; *ledger*: application-layer L_1 plus the basis ledger and a rebooking compensator. The vanilla/validated/sequential rows of the first three blocks are the committed September run; the verified, ledger and deepseek-chat rows are an October run of all four arms (its own vanilla/validated arms: $A_1 = 94$, 44; residue 12, 51) executed as four sequential blocks of 100 sessions. Every R cell is re-derived from the records (`a7_rescore --check`); the ledger rows' R is read off the compensation record and is definitional, so only their outcome column measures anything. Pinned read-to-commit intervals: supplement § W.

Model	Mode	A_1 (95% CI)	R	Booking ≠ final	Regen.
gpt-4o-mini	vanilla	60/100 [49.7, 69.7]	40	100/100	0
	validated (L_1)	0/100 [0, 3.6]	34	34/100 [24.8, 44.2]	66
	verified (L_1, verified store)	0/100 [0.0, 3.6]	51	51/100 [40.8, 61.1]	49
	ledger (L_1+ℓ)	0/100 [0.0, 3.6]	0	0/100 [0.0, 3.6]	55+45 (45/45 obl.)
	sequential	0/100 [0, 3.6]	100	100/100	0
claude-haiku-4-5	vanilla	91/100 [83.6, 95.8]	9	100/100	0
	validated (L_1)	0/100 [0, 3.6]	7	7/100 [2.9, 13.9]	93
	verified (L_1, verified store)	0/100 [0.0, 3.6]	2	2/100 [0.2, 7.0]	98
	ledger (L_1+ℓ)	0/100 [0.0, 3.6]	0	0/100 [0.0, 3.6]	95+5 (5/5 obl.)
	sequential	0/100 [0, 3.6]	100	100/100	0
Llama-3.2 (local)	vanilla	1/100 [0.0, 5.4]	99	100/100	0
	validated (L_1)	0/100 [0, 3.6]	100	100/100	0
	sequential	0/100 [0, 3.6]	100	100/100	0
deepseek-chat	vanilla	45/100 [35.0, 55.3]	55	100/100 [96.4, 100.0]	0
	validated (L_1)	0/100 [0.0, 3.6]	63	63/100 [52.8, 72.4]	37
	verified (L_1, verified store)	0/100 [0.0, 3.6]	52	52/100 [41.8, 62.1]	48
	ledger (L_1+ℓ)	0/100 [0.0, 3.6]	0	0/100 [0.0, 3.6]	47+53 (53/53 obl.)

TABLE 3
Replicated, interleaved live study (probe 303 `--interleave --runs 3`): R per run of 100 sessions at the 150 ms offset, the five arms of a session run back to back in a seeded order, and the pooled count (exact 95% intervals and per-run A_1: supplement § W). A_1 fired in 0/300 sessions of every L_1 arm and in 296/300, 180/300 and 113/300 vanilla sessions; booking ≠ final is $A_1 + R$ in every cell. The ledger rows' R is decided from the obligation record emitted at the superseding put; the share of revisions landing after the final commit (wall-clock stamp) equals R in every L_1 cell.

Arm	haiku	gpt-4o-mini	deepseek-chat
vanilla	2/0/2 (4/300)	38/42/40 (120/300)	60/72/55 (187/300)
validated (L_1)	3/1/3 (7/300)	30/36/43 (109/300)	54/73/58 (185/300)
verified (L_1, verified store)	0/1/1 (2/300)	28/34/42 (104/300)	67/67/60 (194/300)
ledger (L_1+ℓ)	0/0/0 (0/300)	0/0/0 (0/300)	0/0/0 (0/300)
ledger-verified	0/0/0 (0/300)	0/0/0 (0/300)	0/0/0 (0/300)

arm, is where the residue moves: against the sequential run (a contrast that differs in date, seeds and design at once, so it bounds the drift without attributing it; p-values nominal), vanilla A_1 is 94 vs. 296/300 on haiku ($p = 0.018$) and 44 vs. 180/300 on gpt-4o-mini ($p = 0.007$; deepseek-chat $p = 0.197$), the L_1 residue 12 vs. 7/300 and 51 vs. 109/300 ($p < 0.001$, $p = 0.013$; deepseek-chat $p = 0.905$), and within the interleaved design the three runs of one arm differ on deepseek-chat alone (validated residue 54/73/58, χ^2 $p = 0.014$; the pooled intervals treat the 300 sessions as one sample, which that test qualifies). And the ledger rows do not move: 0/300 wrong bookings in both ledger arms on every family (exact 95% interval $[0.0, 1.2]$)—on the store, the qualification Section 6.2 measures.

The offset sweep. Figure 2 varies the race instead of fixing it: the date agent starts 0, 150, 500 or 1,500 ms after the flight agent's read and revises in about half the sessions ($p = 0.5$), $n = 100$ per point on the three API families; at 0 ms the harness imposes only the order (the date agent is released at the flight agent's read), no delay: the models' latencies decide where the revision lands. Without a discipline A_1 fires in 39–46 of 100 sessions at 0 ms and in 0 at 1,500 ms, where every revision lands after the commit, while the booking is wrong in every revised session at every offset (44–51). Under L_1 the curve inverts: the residue is 2–17 at 0 ms and 40–52 at 500–1,500 ms, the discarded generations falling from 34–49 to 0. Predicate and residue are the two halves of one quantity, the revision rate, split by where the revision lands relative to the commit; what the remark of Section 3.6 says must remain after release does remain under every commit-time discipline.

Superstep. A second graph places a writer and a summarizer of one reducer-declared channel in one Pregel superstep [23]; the barrier hands both the snapshot at superstep entry, so in 300/300 sessions the summary names the destination the writer had changed: the snapshot-insufficiency shape of Section 3.5 observed live, reported as documented BSP semantics, not a rate (the census finds it in no public graph; under a completion clock Definition 1 fires in 42, 72, 26 of 100).

6.2 The ledger in a world: a consumer, a cancellation, a compensator that fails

The ledger arm's 0/300 is a statement about the final store, under a compensator that could not fail, a workload that revised once and no reader of the first booking. None of that holds in a world, so we measure the same discipline against a simulated one (probe 304): the booking is an irreversible effect in an in-process service that stays active until cancelled; a *consumer*—a downstream reader that sends the itinerary—reads the active bookings a uniform 0–1500 ms after the first release (a harness parameter); the date agent revises *twice*, 150 ms after the flight agent's read and 800 ms after its first put returns; and the world refuses a cancellation or a rebooking with probability 0.1 each, in which case the obligation stays registered and open, retried at the next supersession and only then. Three arms share every session's seed and run interleaved: L_1 alone; L_1 with the ledger and a compensator that drafts the replacement,

cancels every active stale booking, rebooks, and discharges only when every step succeeded—it runs inside the superseding put, in the date agent's thread, so in the ledger arms that put returns only after the compensation's model call; and the same without the cancellation, the compensator of Table 2 seen from the world. Scored at session end *in the world*: sessions with an active booking whose date is not the final one (*wrong*), with two (*double*), with none (*none*), whose consumer acted on a stale date (*c. stale*), and—the column no store can show—in which the ledger reports every obligation discharged *and* the world holds exactly the final booking. Table 4 is the result on three families, the obligation registered inside the store lock (the shipped design; supplement § W.4 has the after-put run and the first compensator). Under L_1 alone the second revision lands after the regenerated commit in 97–99 sessions of 100, so R fires, store and world are wrong, and the consumer acted on a stale date in the same sessions: with two revisions the residue of Table 2 is nearly total. The ledger registers an obligation in every session whose booking committed before a revision landed (all but 0–1 of 100) and R is 0 in all 300 ledger sessions, by construction (the obligation record is what the predicate reads)—and that is where its guarantee ends. At session end the world holds a stale booking in 10–15 sessions per family (beside the new one in 8–13 of them: the cancellation was refused) and no booking in 7–11 (the rebooking was refused). Those shares are the injected refusal rate of 0.1 seen from the other side, less what a later supersession retried: on haiku only the second revision lands after the commit in 97 of 100 (one obligation, 0 retries); on deepseek-chat both land after it in 77, two obligations in 62, the second retrying what the first left open (15 retries). The consumer, reading within 1.5 s of the first release, acted on a booking the final date contradicts in 98–99 sessions, as under L_1 alone; its `at_ms` against the records places those reads: in 18–31 no revision had landed (the booking was current); in 60–74 the latest revision's compensation had not completed—its model call is that window; in 1–2 the stale booking stood because the world had refused its cancellation (obligation open); in 1–10 the consumer saw the then-current booking, which a later revision replaced—a reader that has read cannot be corrected, only notified. Without the cancellation the world is wrong in 99 of 100 and double-booked in 88–95: a ledger that does not withdraw the effect moves nothing in the world.

The last column is what the ledger is for. In the sessions whose every obligation it reports discharged (75–81 of 100, of which 0–1 registered nothing) the world holds exactly the final booking in every one (0 exceptions), and in every session whose obligation it reports open (19–25) the world is wrong or empty (0 exceptions): report and world agree in 300/300 sessions. That is a property of this compensator, not of the ledger—what a correct compensator owes by construction, which the world check establishes and the records cannot—and the first run of the same probe shows the difference (supplement § W.4): its compensator, which drafted the replacement *between* cancelling and rebooking, reported discharge with the original still active in 1, 5 and 10 of the sessions it called clean; report and world agreed in 278/300. R was 0 in both runs: it cannot tell the two compensators apart; the world does, and the lesson is the remark of Section 3.6: a registered obligation buys exactly its compensator's truthfulness. Two further runs bound the rest. With the obligation registered *after* the put returned (the design until 2026-10-16; another day, unpaired) every world column overlaps the atomic run's (world wrong 4–12, none 6–8, consumer stale 93–100, agreement 300/300)—a null by construction, since the probe injects no fault in that gap. The gap is closed by the code, unmeasured; the compensation it precedes takes 1.0–1.2 s (median), and that is where the residue is. With the consumer reading up to 5 s after the first release it acts on the stale booking in 51–67 of 100 (world wrong 11–12, none 7–10, agreement 300/300): the column measures the reader's window against the compensation's latency, not the ledger.

TABLE 4
The ledger in a simulated world (probe 304, compensator `all`, obligation registered inside the store lock), n=100 sessions per arm per family, two revisions per session, cancellation and rebooking each refused with probability 0.1/0.1, consumer delay uniform 0–1500 ms; arms interleaved per session. Columns: R on the records; store ≠ final; world wrong / double / none at session end; consumer acted on a stale date; obligations registered / discharged / left open; sessions with every obligation discharged in which the world holds exactly the final booking (right / discharged). The after-put registration run and the first compensator are in supplement § W.4.

Model	Arm	R	store	wrong	double	none	c. stale	obl.	right/disch.
haiku	L_1	99	99	99	0	0	99	0/0/0	–
	$L_1+\ell$, cancel	0	7	12	12	7	99	103/84/19	81/81
	$L_1+\ell$, no cancel	0	11	99	88	0	99	103/92/11	1/89
gpt-4o-mini	L_1	97	97	97	0	0	97	0/0/0	–
	$L_1+\ell$, cancel	0	12	15	13	10	98	148/123/25	75/75
	$L_1+\ell$, no cancel	0	11	99	91	0	99	135/124/11	1/89
deepseek-chat	L_1	99	99	99	0	0	99	0/0/0	–
	$L_1+\ell$, cancel	0	13	10	8	11	98	161/140/21	79/79
	$L_1+\ell$, no cancel	0	9	99	95	0	99	155/146/9	1/91

6.3 RQ1 across three disciplines: the AutoGen pilot

An instrumentation layer exposes a two-phase `begin`/`commit` API to LLM-driven agents under AutoGen [14]; the Rust runtimes of Section 5.2 sit behind it. Three workloads span the space from concurrent to sequential: *edit-review* (two agents both read `doc` before either writes—overlap forced), *plan-execute* (strictly sequential—overlap forbidden) and *triage* (a three-agent pipeline whose overlap is left to the models); gpt-4o, 100 sessions per cell, three runtimes (run `20260913T0150Z`; the table is in supplement § M). A_1 fires in 100% / 0% / 48% of vanilla sessions on the three workloads (triage replicated three times: 42%, 48% and 52%; pooled 142/300), in 0% everywhere under pessimistic locking, and in 5% / 0% / 8% under default snapshot isolation, whose residual is its no-write bypass (a read-only commit is never validated: the unfaithful emitter of Theorem L1j; SSI mode closes it). Both endpoints are engineered, so this is a sensitivity check of the detector and the runtimes on model-driven traces, not a prevalence; every edit-review witness is a cold-start read, unlike the live study's supersessions. One harness defect bears on the prices: until 2026-10-17 the pilot's tools told the model `COMMITTED` whatever the runtime did, so a refused agent was never told (records unaffected: a dropped operation emits nothing); they now return `REFUSED` under a recorded policy (`--on-abort drop|retry`). The cells here ran with the old tools and are reported as run. Pessimistic locking reaches 0% on edit-review by refusing the reviewer's operation in 99/100 sessions—the anomaly is absent because the work is—and on triage by dropping one of three agents in 97/100; the sessions that are *complete and clean* (no witness, every role that commits in the unguarded cell also commits) are the number to read a prevention claim against, and on triage they favor the unguarded baseline (52) over default-SI (45) and locking (3); whether a dropped contribution or a stale one is worse is the task's call, which is why Section 6.6 prices prevention per sound committed operation. A Claude Sonnet 4.5 rerun of the vanilla column with operation records reproduced the two determinate cells (edit-review

100/100, plan-execute 0/100) and found the triage cell void: the reporter's prompt supplied no bug, gpt-4o invented one and Sonnet wrote "no bug description provided" in 100/100 sessions. The pilot now seeds the reporter with a concrete bug per scenario and records the protocol beside each cell. Its A_1 rate and the six guarded Sonnet cells are scored by the verified detector; the fold withholds them until the scorer gate's receipt is clean, **[pending: runner step 14 rerun]** (supplement § M). The gpt-4o run predates the change and is reported as run.

6.4 RQ1 for L_2: output commit on synthetic cascades and live verdicts

Three arms run the same 1,000 seeded schedules at nominal chain depths 2, 3 and 5: a root commits, a chain of dependents reads the previous cell and commits, one committed transaction is put under review and retracted or approved, and in a quarter of scenarios the review arrives after that transaction has released. The *verified* arm is `l2_exec.rs`; the *unguarded* arm validates reads, releases at commit and flags only the aborted transaction; the *cascade* arm releases at commit and cascades the abort. Each trace is scored under Definition 4, under the surviving-dependent predicate, and by *exposed effects*, effects out at the end although the review asked to withdraw their basis, which reads no flag (full table: supplement § P). Definition 4 fires in 519, 624 and 563 of 1,000 schedules per depth under both baselines and in none under the verified arm; the cascade arm satisfies its own surviving-dependent predicate in every scenario while exposing exactly as many effects as no discipline at all (1,538, 2,059 and 2,010), because it flagged 767, 1,288 and 1,239 dependents *after* their effects were out—a cascade without output commit moves flags, not effects. The verified arm is not zero on exposed effects: 402, 547 and 549 effects in 199 scenarios per depth, exactly those in which the review arrived after the release and the retraction was refused. The figures decompose by arithmetic ($1{,}538 = 402 + 572 + 564$ at depth 2): the first term no runtime can remove; the second any runtime that declines to release an operation under review avoids; the third (37–46% of the unguarded exposure) is output commit's own contribution, at a cost in delay, not liveness. The quarter of late reviews is a harness parameter, so the overall exposure reduction (73–74%) is that parameter seen from the other side, not a finding.

Live verdicts. A plan–execute–revise workload driven by real models (planner, executor acquiring the planner as predecessor, supervisor deciding retraction) was recorded for 200 sessions on each of three families and replayed through the three arms. Retraction rates varied sharply (1/200, 3/200, 62/200 on claude-haiku-4-5, gpt-4o-mini, Llama-3.2), so only the pooled statement is made: in every one of the 66 retracted sessions the two baselines left a Definition 4 witness and the verified runtime released no executor effect on a retracted plan (pooled release rate 89.0%); 1,000 further haiku sessions produced no retraction, leaving the hold as the discipline's only observable cost there (median 687 ms, p90 789 ms per kept effect). The zero is structural; the A_3 hazard is the backbone's retraction rate.

6.5 RQ1 for L_3: a released tool executor reorders

LangGraph's `ToolNode` dispatches a turn's tool calls concurrently (`asyncio.gather`), collects results in input order and does not serialize side effects (its tracker records a user unable to obtain sequential execution for stateful tools, langgraphjs #861). We gave three backbones a natural prompt over five order-dependent infrastructure tools whose docstrings state their dependencies, took the model-emitted call list as *io*, and dispatched it through the released `ToolNode` and through the L_3 sequencer. Two things are real—whether a backbone batches an order-dependent pipeline into one turn, and the executor—and per-tool latency is modeled, drawn uniformly from 10–200 ms; the replay against real effects below removes the model. The finding that survives the modeling is structural (table in supplement § P): gpt-4o-mini batches an order-dependent pipeline on 30/30 turns, the full five-tool pipeline on 18 of them, and `ToolNode` commits 22 of the 30 out of order (0 under L_3); Llama-3.2 batches 3/30 and reorders 2; claude-haiku-4-5 never batches (0/30); and the transcript a developer inspects shows the correct order in all 33 batched turns while the external state does not in 24 of them. Calls emitted in one turn are, by the provider's contract, calls the model declared parallelizable, so the defect is shared between a backbone that misjudged independence and an executor that cannot notice; the L_3 sequencer makes effect order independent of the backbone. Replaying the 33 recorded batches five times each through `ToolNode` against real effects—a WAL-backed SQLite database and a local HTTP service, with whatever latency those have—rather than a modeled sleep, baseline A_6 fires in 126/165 replays (SQLite) and 112/165 (HTTP): the reorder is a property of concurrent dispatch under any real latency, not of the distribution we drew. The L_3 arm of that replay issued the calls one at a time, which cannot reorder and measures nothing; the arm that counts dispatches the batch concurrently through the same executor and releases each real effect only when the verified sequencer (Section 5.4) releases it. In a second, separate replay of the same 33 batches (runner step 15, with its own baseline arm, which differs from the first run's because concurrent dispatch is nondeterministic): A_6 in 0/165 replays (SQLite; baseline 119/165 in that run) and 0/165 (HTTP; baseline 113/165).

L_4: validation at dispatch. One seeded schedule per run pins a tool and then revokes or re-signs random tools; three dispatchers act on the same registry. Over 1,000 runs at registry widths 2, 4 and 16 the churn touched the planned tool in 630, 505 and 387 runs, and in exactly those both live and snapshot resolution called a phantom (Theorem L_4d, measured) while the verified dispatcher called none and refused exactly those runs; the informative figures are the hazard rate and the refusal cost, the zero is Theorem L_4b executing.

6.6 RQ2: what prevention costs

The pilot driver captures per-call usage and wall-clock for every session of run `20260913T0150Z` (gpt-4o, 900 sessions) and of its Claude Sonnet 4.5 replication (May 2026, no operation records; Section 6.3), at list price at the time of the run. Per session, snapshot isolation's token cost is not separable from the unguarded runtime's on either provider (overlapping intervals in every cell); pessimistic locking's is provider-dependent (at most 1.6× on gpt-4o and 2.2× on Claude, both on triage). A per-session denominator credits a discipline for work it declined to do, so Table 5 re-prices the same run per committed operation and per *sound* committed operation—one that is not the stale reader of any Definition 1 witness.

Parity does not survive the change of denominator: snapshot isolation costs +7.4%, +0.4% and +11.1% per sound committed operation, and pessimistic locking's triage penalty is +97% rather than +43%. Overhead tracks realized contention: edit-review, which aborts hardest, pays most; plan-execute, which never aborts, pays at most +1.7% under any denominator. A separate 155-

TABLE 5
Token cost of prevention under three denominators, gpt-4o, run `20260913T0150Z`. Per sound committed operation is the price we recommend reading. Underlying counts (commits / sound): edit-review vanilla 212/112, pessimistic 101/101, SI 110/105; triage vanilla 335/283, pessimistic 205/205, SI 268/256; plan-execute ≈ 200/200 for all three.

Workload	Discipline	per session	per commit	per sound commit
edit-review	pessimistic	+3.8%	+117.9%	+15.1%
edit-review	SI	+0.6%	+94.0%	+7.4%
plan-execute	pessimistic	+1.7%	+1.2%	+1.2%
plan-execute	SI	+0.4%	+0.4%	+0.4%
triage	pessimistic	+42.7%	+133.2%	+97.0%
triage	SI	+0.5%	+25.6%	+11.1%

session paired sweep on gpt-4o-mini (fan-in 2–16, realized abort rates 0.09–0.93) resolves the mechanism: snapshot isolation carries no fixed overhead (intercept +0.05%, CI $[-1.5, +1.7]$) and pays $1.074\times$ $[1.052, 1.096]$ a first-attempt generation per abort, because the retry carries the conflict into its prompt (supplement § M).

Wall-clock, with its confound. The nine cells of a run ran one after another, so a cell difference carries the provider's latency drift. Plan-execute, on which no runtime aborts, is the control; its "overheads" are −4.5% and −4.9% on gpt-4o (Holm-corrected permutation $p = 0.014$, 0.002) and +13.5% for SI on Claude ($p = 0.001$): drift, whose largest magnitude is the run's null band. Outside it: pessimistic locking at +16.7% (edit-review) and +34.9% (triage) on gpt-4o and +109.5% (triage) on Claude, 7.75 model calls per session against 4.00. Snapshot isolation is inside the band in every cell on both families—not zero: the design resolves nothing below about 5% on gpt-4o or 14% on Claude (supplement § O). Probe 302 removes the confound by interleaving the runtimes per session. Its Claude Sonnet 4.5 run (900 sessions, runtimes shuffled per session) lost its per-call sidecars to a script defect and is read from completion stamps at 1 s resolution (supplement § O): triage pessimistic +1.3% [-1.1, +3.7], $n = 99$, the plan-execute control −0.5% [-3.5, +2.5], $n = 99$, edit-review pessimistic +5.3% [+2.0, +8.4], $n = 99$. That cell and the +109.5% above are two readings of a pilot that told the refused agent `COMMITTED` (Section 6.3): 7.75 chat calls per session in May, 4 in October (supplement § M). With the tools telling the truth (Claude Sonnet 4.5, five arms interleaved per session, n=100, edit-review and triage) the price is what the lie had hidden, and the sidecars say who pays it. Under locking the triager's read-only operation holds the ticket's lock from one of its turns to the next, so the reporter and the engineer are refused on every turn of theirs (8.47 refused operations per session) and the chat runs on: told `REFUSED`, 15.01 model calls per session against vanilla's 4.00, wall-clock +297.8% [+278.4, +315.4], $n = 100$; asked to redo, 16.98 calls, +332.7% [+324.8, +340.4], $n = 100$. On edit-review the editor's lock refuses the reviewer five times per session (11.00 calls against 5.00, +104.2% [+99.8, +108.7], $n = 100$), and the editor never issues its own write (0/100 sessions against 100/100 unguarded; 0 of its 1,100 completions have a `write_state` call's length, against 147 of 500), re-reading in each of its six turns. The chat shows the reviewer's `REFUSED` result to the editor too, and the editor acts on it: under snapshot isolation, which refuses only the reviewer's overwritten commit (44/100 sessions; +19.7% [+13.8, +25.7], $n = 100$ drop, +52.0% [+38.3, +66.1], $n = 100$ retry; never on triage), the editor then repeats a step of its own, a blind rewrite or a re-read (1.44 records per session against 1.00), while under the earlier tools, whose refused read surfaced as an exception text (the 2026-10-14 run), the locked editor wrote in 100/100 sessions. The control addresses the `REFUSED` text to the refused agent and marks it as not theirs to the others (runner step 24; four of the five arms, n=100; supplement § O). On triage it removes most of the locking price: told to drop, 4.30 calls against vanilla's 4.00 (1.10 refusals per session, +7.9% [+3.3, +13.1], $n = 100$), at 2.10 committed operations per session against 3.00—the refused agents' operations are dropped, not repeated; told to retry, 9.00 calls, 3.87 commits, +142.6% [+113.9, +171.8], $n = 100$. On edit-review it changes nothing: 11.00 calls, 5.00 refusals, the editor's write in 0/100 sessions and 0 write-length completions of 1,100, under both texts. The price of a truthful refusal is thus set by what the chat shows the unrefused agent; its wording decides it on one workload and not on the other, and which feature of the `REFUSED` result costs the editor its write these runs do not isolate (the one text under which the locked editor wrote is the exception text of the 2026-10-14 run).

6.7 RQ3: where the preconditions occur

To put a denominator under the structural precondition of A_1 in the framework we studied—a reader and a writer of one last-value channel released into the same superstep—we analyzed public code; the answer is *none confirmed*, by an instrument blind to about half the repositories. *Frame*: 60 GitHub repositories per star band matching `langgraph` (Python; forks and the framework's own organization excluded), a seeded draw from the API's 500 most recently updated; all 180 were read (supplement § Q). *Instrument*: a static front end that recovers from the syntax tree each `StateGraph` site, the node pairs one fork releases into a superstep, and each node's read and written keys; what it cannot resolve is *unknown*, never guessed (validated against execution on eight runnable fixtures). Of the 180, 98 contain a visible graph, holding 407 sites, 392 (96%) of them sequential. Fifteen sites in 11 repositories (11.2%, Wilson 95% $[6.4, 19.0]$) contain a co-scheduled pair; none is a confirmed reader/writer pair on a last-value channel (repository-level upper limit 3.8%); two are such pairs on a reducer channel (the parallel-agents-on-`messages` idiom); eight have an unresolved side, bounding the susceptible share at 8.2% $[4.2, 15.3]$ of repositories. Counting every site the front end cannot settle—an unresolved side, `Send` fan-out (seven sites, the construct outside the model, Section 2) or routing it cannot resolve—gives 94 sites in 50 repositories (51%): the instrument's reach, where nothing is confirmed or excluded, not a bound. An enriched population—the 34 third-party repositories whose issues quote LangGraph's co-superstep writer/writer error, so demonstrably ran parallel writers—has parallel graphs in 12 of its 26 with a visible graph but again no confirmed site in 391. Of five deployed designs we probed, four close the window by construction; LangGraph's superstep channels and cross-thread store leave it open, where Section 6.1 measures it (supplement § Q).

6.8 Threats to validity

Construct. A_1 tracks the window, not the harm; Table 2 reports both and R names the gap. The basis is argument provenance, verbatim and not semantic; whether a booking that disagrees with a later revision is harm is the application's call, and we count it because the user asked for one trip. *Internal.* The live rates are latency races, $n = 100$ per cell, single runs in Table 2 and Figure 2 and three interleaved runs in Table 3; the application-layer L_1 arm is

check-then-put (Section 6.1); the sequential wall-clock cells carry a run-order confound we bound rather than remove (Section 6.6); the pilot's prices depend on what a model does with a refusal, its own or a peer's broadcast one (Section 6.6); RQ2 is priced in tokens for L_1 only—L_2 in hold latency, L_4 in refused dispatches, L_3 unpriced (the sequencer adds no model call); the L2 live replay rests on 66 retractions, 62 from one family. *External.* Three API families and one local; one framework driven live; toy workloads of two to three agents. *Reproducibility.* Every cell is committed with its traces and rescored by the verified detector and an independent implementation of Definition 1; one September run has a run-time manifest, two reconstructed ones that say so.

7 Related work

7.1 Classical consistency and isolation

The hardware tradition [5], [6] characterizes visibility under bounded latency; ours is unbounded. The database isolation hierarchy [7]–[9] is the direct ancestor: we adopt its anomaly-based method over a different operational model, and the SSI line [12], [20], [25] is the reference for our SSI runtime. Commitment ordering [77], Weikum and Vossen's commit-order-preserving serializability [3] and Kung and Robinson's backward validation [4]—the read-set validation at commit of Silo [75] and TicToc [76], which L_1 is—are what Theorem 1 mechanizes at the level of values read (the value-level statement differs from the conflict-level one exactly at the overwrite-and-restore shape of Section 3.1); view equivalence [26] is where the value-level and conflict-level notions meet, and Definition 2 is a client-centric, state-based statement of the kind Crooks et al. give for isolation levels [74]. Highly available transactions [62] classify levels by admitted anomalies, the method our lattice borrows; Cerone et al. [27] give the axiomatic framework our predicates specialize; *robustness* [28] is what the census asks statically. Compensation descends from sagas [17], its formal treatment [19], ConTracts [44] and Gray and Reuter [50]; recoverability and strictness are Hadzilacos [42] and Bernstein et al. [16]. Irrevocable transactions [29] and transactional boosting [51] met the same irreversibility in transactional memory; output commit is the dual choice. Commutativity-based control [63] orders only what does not commute: an A_6 discipline over commutativity classes would pay the sequencing cost only where it buys something. The basis of Definition 6 is why-provenance [64] restricted to copying; exactly-once stream processing [66] holds side effects at a transactional sink until the input is durable, the industrial precedent for output commit. The pessimistic/optimistic choice is STM's contention-management problem; cost-aware level selection is consistency rationing [52] and TxnSails [49].

7.2 Concurrency control and shared memory for LLM agents

A 2025–2026 line builds mechanisms for this regime, mostly without verifying them, and this paper is one point in it rather than its origin; twenty of the 2026 works cited here are preprints at the time of writing (arXiv identifier, no venue), cited for the designs they describe—no claim of ours rests on their results. Agentic transactions [67] recast ACID as four semantic guarantees for one long-horizon agent and measure task success, with no history predicate, model or proof; our catalog is the anomaly-level account such guarantees would be checked against (supplement § D). CoAgent [36] states the regime in our three terms and proposes an advisory protocol in which a notified agent repairs its own plan, correct conditional on a model judgment measured at a 5% failure rate; its lock/OCC cost inversion independently reproduces Section 6.6. STORM [37] is read validation (L_1) deployed per file; ATCC [39] and GoEX [40] impose a remedy from the runtime. MemTX [53] is the closest design to our L_2: staged writes under snapshot isolation, irreversible tool calls gated on current belief, cascading repair on retraction, two invariants checked by bounded enumeration (5.5M states), five backbones; it does not state the predicates it excludes, verify the runtime, or price prevention per unit of sound work. Yang et al. [54] argue that coordination failures attributed to communication are concurrency failures; governed shared memory [55] and transactive memory [56] address provenance, supersession and scoping without treating the generation window as a transaction. TOKI [69] is the closest formal treatment of supersession: contradiction resolution in agent memory as write-time concurrency control, with a soundness proof for a bitemporal operator family; it governs which belief a memory keeps, where R governs what an effect already out still owes. SAFEFLOW [70] adds information-flow labels and write-ahead rollback to transactional execution, and Cordon [71] stages external actions in an effect outbox released after whole-workflow validation—output commit at task granularity, without a model of concurrent tasks or a proof. StateFuse [72] keeps conflicting observations visible under CRDT merging, orthogonal to exclusion like CodeCRDT; Yu et al. [73] ask whether consistent shared memory suffices for an agent's task duties, a question above our history level. SagaLLM [1], Atomix [2] and CodeCRDT [24] are placed on the lattice informally in supplement § D (none reaches L_4; § V compares the nearest systems dimension by dimension). S-Bus [38] (the author's prior work) recovers implicit read sets for one contract, the natural supplier of trust-base item (ix); frameworks [14], [68] supply substrate, not contract.

7.3 Commit-time guards, retraction, and what comes after commit

The 2026 work on agent effects acts at the commit boundary: CommitGuard [58] licenses a durable effect only by a witness still fresh at commit; Zheng et al. [59] show that freshness guards of our L_1 kind block 92–95% of benign infrastructure races and argue for semantic commit predicates; EffectMatch [60] validates persistent outcomes against what was approved; versioned execution [61] revokes an obsolete execution's authority to publish. MemTX [53] alone looks past commit: on an explicit *retraction* it walks a derivation DAG to executed tool actions and records each as compensated or leaked. None treats the ordinary case Table 2 measures—no retraction, no race, a legitimate later write—or says what is achievable there; Section 3.6 says an obligation, registered at the superseding write. Zheng et al.'s precision objection is met at the basis, as far as the basis is recovered: a basis predicate ignores writes the effect never depended on.

7.4 Effect order, durable execution, verified systems, taxonomies

Parallel function calling is pursued deliberately (LLMCompiler [34], asynchronous calling [57]), the ordering of effects left to the runtime; A_6 is a wrong independence judgment seen at the effect level (Section 6.5). Durable-execution engines [35] and exactly-once workflow systems [43] govern whether an effect is *repeated* after a crash (the property D of Section 3.7 names), output

commit whether it may *leave* while its basis can be withdrawn; the two compose. Rollback recovery discharges duplicates by identity because replay is deterministic, a premise agent runtimes break [48]. IronFleet [30] and Perennial [45] verify running systems by layered refinement, the template for the open step of Section 5.2; ours add verified bytes executed in a measurement. Jepsen [31] and the black-box checkers [32], [33] (successors in supplement § R) decide isolation levels over database histories with unverified analyzers under a uniqueness assumption we share (Section 3.3); ours decide a far simpler property, verified, in the tradition of runtime verification [46]. MAST [11] catalogs cognitive and coordination failures; its traces cannot show A_1 either way (supplement § C). Agentproof [41] analyzes agent graphs statically, as the census does; MCMAS [47] verifies task-level and strategic properties.

8 Limitations and Future Work

The predicates are coarser than the phenomena. A_1 over-reports in one shape (a value overwritten and restored inside the window; 0 of 1,299 witness pairs on the live records) and under-reaches the harm by the post-commit supersession R names (Table 2); A_6 fires on commuting effects and is blind to dropped and duplicated ones; Table 1 says where specification and definition coincide, under what hypothesis.

R is not an anomaly, and its basis is syntactic. R fires in $100/100$ sequential sessions because the workload revises the date in every session; Table 2 establishes that the harm an isolation discipline cannot reach is exactly this predicate, not that concurrency causes it. Its basis is argument provenance: a booking derived from the date by computation has an empty basis and is invisible, so the R column is a lower bound (coverage 9,000/9,000 here, the date being a structured argument); free-text effects need a declared or taint-tracked basis.

A refused retraction is not a prevented anomaly. Output commit makes Definition 4 unreachable partly by refusing to retract an operation whose effects are out, which leaves the world as it was and is counted (Section 6.4); the exposure–latency curve between harness and live driver is unmeasured.

Verification is relative to the trace, and the triangle closes on one system only in part. Trust-base item (ix) is the largest, measured only at the store boundary: a value that reaches the model through prompt context is invisible, and our own read-set reconstruction finds most reads invisible on a realistic workload [38]. The A_1 verdict is the verified bytes behind an interning wrapper, its gate receipt pending; A_2, A_3 and A_6 remain ports gated against their specifications. The refinement from the two concurrent stores to their abstract machines is open (a bounded simulation is evidence, not proof); the application-layer L_1 arm is check-then-put, its window bounded by 3,900 sessions, not closed (Section 6.1); the L_2–L_4 exec artifacts are sequential objects a multi-threaded caller must serialize; the ledger arm's executable is harness code except its bookkeeping (10 obligations, 0 errors), its Verus model scaffolding. Liveness is verified nowhere.

Prevalence is not established, and the census points the other way. The census bounds a structural precondition in public code (none confirmed; routing unresolved in about half the repositories) and is silent on private deployments, run frequency and the cross-thread store.

Pending (runner steps): the clean receipt of the verified-scorer gate (step 14; its last pass rescored every cell but exited non-zero on the refusal study's per-cell sidecars); the Claude pilot's pessimistic and snapshot-isolation cells and the A_1 rate of its rerun triage cell (11, 14); and no other. Longer tasks, more agents and an instrumented deployment are the empirical follow-up; the store refinements, the formal one.

9 Conclusion

What commit-time isolation leaves is not an anomaly: an effect out on a basis later writes revise, which under an open-world store can only be owed, and an obligation is worth exactly what its compensator does.

What an agent framework should provide. Four requirements follow from the measurements, each priced. (i) *A versioned store with compare-and-set.* LangGraph's `BaseStore` has neither, so L_1 cannot live inside the framework; with it, read-set validation removes every stale commit at one discarded generation per realized conflict, and per sound commit optimistic validation costs $+0.4$–11.1% where locking costs up to $+97\%$. (ii) *An obligation hook on the write path*, atomic with the write, and compensators tested against the external system, not the records: R was 0 under a compensator that misreported, and the stale effect stands for the compensation's model call (median 1.0–1.2 s). (iii) *A sequencer in the tool executor* for calls emitted in one turn unless declared commuting: the released `ToolNode` committed $126/165$ and $112/165$ replays out of order against real effects, the verified sequencer none. (iv) *Validation at dispatch* for tools of a mutable registry, and an idempotency key on resumed effects: snapshot resolution called a phantom tool in every run the churn touched; a re-executed node emitted its effect twice.

References

[1] E. Y. Chang and L. Geng, "SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning," *Proc. VLDB Endow.*, 2025.

[2] B. Mohammadi, N. Potamitis, L. Klein, A. Arora, and L. Bindschaedler, "Atomix: Timely, transactional tool use for reliable agentic workflows," *arXiv:2602.14849*, 2026.

[3] G. Weikum and G. Vossen, *Transactional Information Systems*. Morgan Kaufmann, 2001.

[4] H. T. Kung and J. T. Robinson, "On optimistic methods for concurrency control," *ACM Trans. Database Syst.*, vol. 6, no. 2, pp. 213–226, 1981.

[5] L. Lamport, "How to make a multiprocessor computer that correctly executes multiprocess programs," *IEEE Trans. Comput.*, vol. C-28, no. 9, pp. 690–691, 1979.

[6] M. P. Herlihy and J. M. Wing, "Linearizability: A correctness condition for concurrent objects," *ACM TOPLAS*, vol. 12, no. 3, pp. 463–492, 1990.

[7] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, "A critique of ANSI SQL isolation levels," in *ACM SIGMOD*, 1995, pp. 1–10.

[8] A. Adya, "Weak consistency: A generalized theory and optimistic implementations for distributed transactions," Ph.D. dissertation, MIT, 1999.

[9] A. Adya, B. Liskov, and P. O'Neil, "Generalized isolation level definitions," in *Proc. ICDE*, 2000, pp. 67–78.

[10] W. Lloyd, M. J. Freedman, M. Kaminsky, and D. G. Andersen, "Don't settle for eventual: Scalable causal consistency for wide-area storage with COPS," in *SOSP*, 2011, pp. 401–416.

[11] M. Cemri *et al.*, "Why do multi-agent LLM systems fail?" in *NeurIPS Datasets and Benchmarks Track*, 2025.

[12] M. J. Cahill, U. Röhm, and A. D. Fekete, "Serializable isolation for snapshot databases," in *Proc. SIGMOD*, 2008, pp. 729–738.

[13] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[14] Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.

[15] M. S. Miller, "Robust composition: Towards a unified approach to access control and concurrency control," Ph.D. dissertation, Johns Hopkins Univ., 2006.

[16] P. A. Bernstein, V. Hadzilacos, and N. Goodman, *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987.

[17] H. Garcia-Molina and K. Salem, "Sagas," in *Proc. SIGMOD*, 1987, pp. 249–259.

[18] R. E. Strom and S. Yemini, "Optimistic recovery in distributed systems," *ACM Trans. Comput. Syst.*, vol. 3, no. 3, pp. 204–226, 1985.

[19] H. F. Korth, E. Levy, and A. Silberschatz, "A formal approach to recovery by compensating transactions," in *Proc. VLDB*, 1990, pp. 95–106.

[20] A. Fekete, D. Liarokapis, E. O'Neil, P. O'Neil, and D. Shasha, "Making snapshot isolation serializable," *ACM Trans. Database Syst.*, vol. 30, no. 2, pp. 492–528, 2005.

[21] A. Lattuada *et al.*, "Verus: Verifying Rust programs using linear ghost types," *Proc. ACM Program. Lang. (OOPSLA)*, vol. 7, 2023, pp. 286–315.

[22] Anthropic, "Model context protocol," https://modelcontextprotocol.io, 2024.

[23] G. Malewicz *et al.*, "Pregel: A system for large-scale graph processing," in *Proc. SIGMOD*, 2010, pp. 135–146.

[24] S. Pugachev, "CodeCRDT: Observation-driven coordination for multi-agent LLM code generation," *arXiv:2510.18893*, 2025.

[25] D. R. K. Ports and K. Grittner, "Serializable snapshot isolation in PostgreSQL," *Proc. VLDB Endow.*, vol. 5, no. 12, pp. 1850–1861, 2012.

[26] C. H. Papadimitriou, "The serializability of concurrent database updates," *J. ACM*, vol. 26, no. 4, pp. 631–653, 1979.

[27] A. Cerone, G. Bernardi, and A. Gotsman, "A framework for transactional consistency models with atomic visibility," in *Proc. CONCUR*, 2015, pp. 58–71.

[28] G. Bernardi and A. Gotsman, "Robustness against consistency models with atomic visibility," in *Proc. CONCUR*, 2016, pp. 7:1–7:15.

[29] A. Welc, B. Saha, and A.-R. Adl-Tabatabai, "Irrevocable transactions and their applications," in *Proc. SPAA*, 2008.

[30] C. Hawblitzel *et al.*, "IronFleet: Proving practical distributed systems correct," in *Proc. SOSP*, 2015, pp. 1–17.

[31] K. Kingsbury, "Jepsen," https://jepsen.io, 2013–present.

[32] C. Tan, C. Zhao, S. Mu, and M. Walfish, "Cobra: Making transactional key-value stores verifiably serializable," in *Proc. OSDI*, 2020, pp. 63–80.

[33] K. Kingsbury and P. Alvaro, "Elle: Inferring isolation anomalies from experimental observations," arXiv:2003.10554, 2020.

[34] S. Kim *et al.*, "An LLM compiler for parallel function calling," in *Proc. ICML*, 2024.

[35] S. Burckhardt, C. Gillum, D. Justo, K. Kallas, C. McMahon, and C. S. Meiklejohn, "Durable Functions: Semantics for stateful serverless," *Proc. ACM Program. Lang. (OOPSLA)*, vol. 5, 2021.

[36] H. Lyu, D. Zhang, M. Wu, X. Wei, and H. Chen, "CoAgent: Concurrency control for multi-agent systems," arXiv:2606.15376, 2026.

[37] M. Liu, T. Chen, Z. Xu, X. Jiang, and Y. Dong, "Multi-agent collaboration with state management," arXiv:2605.20563, 2026.

[38] S. Khan, "S-Bus: Automatic read-set reconstruction for multi-agent LLM state coordination," arXiv:2605.17076, 2026.

[39] W. Zhou, Z. Wang, Z. Peng, H. Chen, Y. Zhang, and G. Yu, "ATCC: Adaptive concurrency control for unforeseen agentic transactions," arXiv:2603.13906, 2026.

[40] S. G. Patil *et al.*, "GoEX: Perspectives and designs towards a runtime for autonomous LLM applications," arXiv:2404.06921, 2024.

[41] M. Xavier, Vaisakh M. A., M. Jolly, and M. Xavier, "Agentproof: Static verification of agent workflow graphs," arXiv:2603.20356, 2026.

[42] V. Hadzilacos, "A theory of reliability in database systems," *J. ACM*, vol. 35, no. 1, pp. 121–145, 1988.

[43] A. Skiadopoulos *et al.*, "DBOS: A DBMS-oriented operating system," *Proc. VLDB Endow.*, vol. 15, no. 1, pp. 21–30, 2021.

[44] A. Reuter and H. Wächter, "The ConTract model," *IEEE Data Eng. Bull.*, vol. 14, no. 1, pp. 39–43, 1991.

[45] T. Chajed, J. Tassarotti, M. F. Kaashoek, and N. Zeldovich, "Verifying concurrent, crash-safe systems with Perennial," in *Proc. SOSP*, 2019, pp. 243–258.

[46] A. Bauer, M. Leucker, and C. Schallhart, "Runtime verification for LTL and TLTL," *ACM TOSEM*, vol. 20, no. 4, pp. 14:1–14:64, 2011.

[47] A. Lomuscio, H. Qu, and F. Raimondi, "MCMAS: an open-source model checker for the verification of multi-agent systems," *Int. J. Softw. Tools Technol. Transf.*, vol. 19, no. 1, pp. 9–30, 2017.

[48] Y. Zheng, Y. Yang, W. Zhang, and A. Quinn, "ACRFence: Preventing semantic rollback attacks in agent checkpoint-restore," arXiv:2603.20625, 2026.

[49] W. Zhou, Y. Zhang, Z. Peng, and G. Yu, "TxnSails: Achieving serializable transaction scheduling with self-adaptive isolation level selection," arXiv:2502.00991, 2025.

[50] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993.

[51] M. Herlihy and E. Koskinen, "Transactional boosting: A methodology for highly-concurrent transactional objects," in *Proc. PPoPP*, 2008, pp. 207–216.

[52] T. Kraska, M. Hentschel, G. Alonso, and D. Kossmann, "Consistency rationing in the cloud: Pay only when it matters," *Proc. VLDB Endow.*, vol. 2, no. 1, pp. 253–264, 2009.

[53] X. Li *et al.*, "MemTX: Transactional belief commit for stateful agent memory," arXiv:2607.23929, 2026.

[54] X. Yang, L. Li, Z. Ji, T. J. Zhang, and W. Jiang, "Position: Multi-agent systems should prioritize concurrency control," arXiv:2608.18092, 2026.

[55] Y. Margalit, N. Cohen-Inger, E. Avram, R. Taig, and O. Margalit, "Governed shared memory for multi-agent LLM systems," arXiv:2606.24535, 2026.

[56] T. E. Kim, X. He, D. Jain, A. Agrawal, N. Arabzadeh, and F. Diaz, "Multi-agent transactive memory," arXiv:2606.19911, 2026.

[57] I. Gim, S. Lee, and L. Zhong, "Asynchronous LLM function calling," arXiv:2412.07017, 2024.

[58] I. Santos-Grueiro, "Temporary authority, permanent effects: Commit-time authorization for LLM agents," arXiv:2607.10487, 2026.

[59] Z. Zheng, J. Long, B. Li, and J. Yao, "Stale does not mean unsafe: Guard precision for tool-using LLM agents under infrastructure state races," arXiv:2609.29522, 2026.

[60] H. Zhang, H. Zhang, Z. Liang, Y. Yan, D. Zuo, and H. Wang, "Beyond approved actions: Runtime validation of persistent outcomes in agent workflows," arXiv:2609.31301, 2026.

[61] Y. Zhang, R. Sharma, N. Vegesna, Z. Fu, C. Liu, and T. Krishnamurthy, "Serving a revisable world: Versioned execution for interruptible agents," arXiv:2610.01160, 2026.

[62] P. Bailis, A. Davidson, A. Fekete, A. Ghodsi, J. M. Hellerstein, and I. Stoica, "Highly available transactions: Virtues and limitations," *Proc. VLDB Endow.*, vol. 7, no. 3, pp. 181–192, 2014.

[63] W. E. Weihl, "Commutativity-based concurrency control for abstract data types," *IEEE Trans. Comput.*, vol. 37, no. 12, pp. 1488–1505, 1988.

[64] P. Buneman, S. Khanna, and W.-C. Tan, "Why and where: A characterization of data provenance," in *Proc. ICDT*, 2001, pp. 316–330.

[65] A. Gupta and I. S. Mumick, "Maintenance of materialized views: Problems, techniques, and applications," *IEEE Data Eng. Bull.*, vol. 18, no. 2, pp. 3–18, 1995.

[66] T. Akidau *et al.*, "MillWheel: Fault-tolerant stream processing at internet scale," *Proc. VLDB Endow.*, vol. 6, no. 11, pp. 1033–1044, 2013.

[67] Z. Sun, X. Wang, and G. Li, "Agentic transaction: Towards ACID-compliant agent systems," arXiv:2608.13900, 2026.

[68] LangGraph 1.2, Letta, CrewAI 1.15, GitHub repositories, 2026.

[69] Z. Wang, "TOKI: A bitemporal operator algebra for contradiction resolution in LLM-agent persistent memory," *arXiv:2606.06240*, 2026.

[70] P. Li, Z. Wu, Y. Zhang *et al.*, "SAFEFLOW: A principled protocol for trustworthy and transactional autonomous agent systems," *arXiv:2506.07564*, 2025.

[71] Z. Chen *et al.*, "Cordon: Semantic transactions for tool-using LLM agents," arXiv:2606.17573, 2026.

[72] S. Volkov, Y. Li, and Y. Luo, "StateFuse: Deterministic conflict-preserving memory for multi-agent systems," *arXiv:2607.05844*, 2026.

[73] Z. Yu, Z. Wang, P. Wang, H. Lin, R. Zhao, and C. Qin, "Beyond corrected memory: Execution consistency in multi-agent systems," *arXiv:2610.08101*, 2026.

[74] N. Crooks, Y. Pu, L. Alvisi, and A. Clement, "Seeing is believing: A client-centric specification of database isolation," in *Proc. PODC*, 2017, pp. 73–82.

[75] S. Tu, W. Zheng, E. Kohler, B. Liskov, and S. Madden, "Speedy transactions in multicore in-memory databases," in *Proc. SOSP*, 2013, pp. 18–32.

[76] X. Yu, A. Pavlo, D. Sanchez, and S. Devadas, "TicToc: Time traveling optimistic concurrency control," in *Proc. SIGMOD*, 2016, pp. 1629–1642.

[77] Y. Raz, "The principle of commitment ordering," in *Proc. VLDB*, 1992, pp. 292–312.

[78] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[79] D. Cousineau, D. Doligez, L. Lamport, S. Merz, D. Ricketts, and H. Vanzetto, "TLA+ proofs," in *Proc. FM*, 2012, pp. 147–154.